Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.
Digital Object Identifier 10.1109/ACCESS.2017.DOI

Benchmarking Deep Reinforcement Learning for Cartesian Path Tracking of Free-Floating Space Robot Manipulators

PATRICK S. ERMISCH[1], and ERIC GUIFFO KAIGOM[1]
[1]Department of Computer Science & Engineering, Frankfurt University of Applied Sciences, Frankfurt am Main, Germany
(e-mail: steven.ermisch@stud.fra-uas.de, kaigom@fra-uas.de)
Corresponding author: Eric Guiffo Kaigom (e-mail: kaigom@fra-uas.de).

ABSTRACT Free-floating space robots exhibit a strong dynamic coupling between the manipulator motion and base attitude disturbance, complicating the Cartesian path tracking of the end-effector. Supervised data-driven approaches are challenging due to limited labeled data availability in space operations. This paper therefore explores trial-and-error learning techniques for Cartesian path tracking in a physics-based Simulink/Simscape environment with integrated safety monitoring. Since a quantitative and qualitative comparison of these methods is currently missing in the sector of on-orbit servicing, we benchmark six Reinforcement Learning agents: TRPO, TD3, SAC, PPO, Policy Gradient, and DDPG. We evaluate their performance using unified key performance indexes. These include training duration and stability, tracking accuracy, base attitude disturbance, motion smoothness, energy consumption, and collision risks. We observe that PPO achieves the best overall trade-off between tracking accuracy, base disturbance minimization, and training stability. Building on this insight, we quantify the effect of a systematic tuning of the neural network architecture and key hyperparameters of PPO for tracking both circular and ramp-shaped Cartesian paths. Compared to default PPO configuration, the optimized version significantly improves the mean episodic return by 80.7%, reduces the mean squared tracking error by 67.7% and the maximum tracking error by 44.1%, and stabilizes the base orientation (67.0% reduction). Moreover, the optimized PPO eliminates collisions and joint-limit violations while reducing the energy consumption by 15.98%.

INDEX TERMS Free-floating on-orbit servicing, Proximal Policy Optimization (PPO), reinforcement learning, safe simulation, space robotics, trajectory tracking

I. INTRODUCTION AND BACKGROUND

Free-floating robot manipulators operating in the space environment to inspect, capture, or de-orbit satellites are expected to introduce unique control challenges when compared to terrestrial robots [1]. In micro-gravity, the robot's base (typically a spacecraft) is not fixed and thrusters are turned off. Manipulator motions can give rise to a change of the position and orientation of the base, which in turn modifies the final pose of the end-effector. This reaction is due to the conservation of the linear and angular momentum [2]. The free-floating dynamics of space robots has many benefits. An expensive consumption of fuel, such as hydrazine (N_2H_4), and an excitation of the robot dynamics for some activation frequencies of the pulse width modulation related to thruster control can be avoided [3]. This has the potential to extend the lifetime of space robots and enhance the mission success.

However, a free-floating base complicates the achievement of robotized manipulations. Indeed, the end-effector of the arm may fail to reach the desired target point if standard motion planning approaches dedicated to robots with a fixed base are simply applied. Further constraints, such as communication latency and limited onboard computational resources, make more intricate the continuous real-time control and stabilization of the base [1], [4]. In fact, traditional control approaches that assume an accurate model of the robot dynamics (e.g. reactionless planning or precise attitude control) can struggle to adapt to uncertainties and unmodeled dynamics. Supervised data-driven approaches, on the other hand, are difficult to apply. Usually, the required amount and quality of data are rather unavailable in the space sector. In many cases, robotized tracking capabilities under a free-floating behavior of the arm's base need to be demonstrated long before the physical robot system is assembled and deployed [5]–[7]. These challenges motivate the development and simulation of AI-empowered digital twins [8] under desired operational conditions to explore learning-based control methods that can autonomously adapt to complex and unknown dynamics even in the case of data scarcity.

Reinforcement Learning (RL) has emerged as a promising approach for robotics, enabling agents to learn control policies through trial-and-error interactions with entities, environments, and processes. Recent advances in deep RL have shown success in complex control tasks, but applying RL to space robots is still relatively novel [9], [12]–[14], [16].

The work presented in this paper investigates the use of continuous-action deep RL for the tracking of a Cartesian motion of the end-effector of a free-floating space manipulator. It differs from previous contributions in three ways. First, it focuses on a systematic comparison and explores the respective performance of six different RL algorithms beyond those mentioned e.g. in Table 1, thereby quantitatively and qualitatively contributing to filling in a current gap in space robotics. Second, several constraints, such as collision avoidance, are monitored through formal methods applied from a software-enabled and practical perspective. Finally, this work investigates the minimization of the base attitude in conjunction with constraint monitoring previously mentioned. Recall that the induced base disturbance is omitted in most recent contributions, as highlighted in Table 1. We are not aware of any comprehensive work that compares RL algorithms for the Cartesian path tracking of free-floating space robots while considering the base attitude minimization and unifying constraint monitoring by leveraging formal methods, to the best of our knowledge (see, also Table 1).

[R3.1] As evident from Table 1, every prior contribution evaluates only a single RL algorithm on a specific subtask, making it impossible to draw conclusions about the relative merits of different learning paradigms for free-floating space robot control. Furthermore, no existing work simultaneously addresses three compounding challenges: (i) a multi-algorithm benchmark under strictly unified conditions, (ii) explicit minimization of base attitude disturbance as a co-equal control objective alongside end-effector tracking, and (iii) integration of formal safety verification directly into the RL training loop. The present work is, to the best of our knowledge, the first to close all three gaps at once, providing a reproducible and safety-aware comparison framework that goes substantially beyond the individual studies listed in Table 1.

The key contributions and novelties of this work include:

- A comparative evaluation of RL Agents: We implement and evaluate six state-of-the-art continuous RL algorithms, Trust Region Policy Optimization (TRPO) [17], Twin Delayed Deep Deterministic Policy Gradient (TD3) [18], Soft Actor-Critic (SAC) [19], Proximal Policy Optimization (PPO) [20], vanilla Policy Gradient (PG) [21], and Deep Deterministic Policy Gradient (DDPG) [22], on the same path tracking task of a free-floating robot. This side-by-side comparison under unified conditions reveals which algorithm is likely to deliver the best possible performance when it comes to motion tracking of a free-floating space robot and why.
- An integration of formal safety measures: To meet safety-critical space operation requirements, we incorporate formal methods into the learning loop [2], [23]–[25]. A collision monitoring, momentum conservation check, and formal verification of joint torque limits are embedded in the simulation. This integration ensures the RL agent cannot violate fundamental and critical safety properties (like avoiding collisions or exceeding actuator limits) during training and testing.
- A high-fidelity simulation environment: We develop a high-fidelity MATLAB/Simulink simulation of a free-floating space robot, using a URDF-based multibody model and the Simscape physics engine. The robot, a cube-shaped base with a 4-Degree of Freedom (DOF) robotic arm, is modeled with realistic kinematics, dynamics, and sensor/actuator parameters and features. A dedicated reinforcement learning environment provides the agent with observations and rewards, including a specially designed reward function that encourages accurate end-effector trajectory tracking while penalizing base movement, large joint velocities, and excessive control effort (energy). Through reward shaping and carefully chosen reset conditions, the agent learns to accomplish the task without causing excessive base disturbance or safety violations [22], [26]–[29].
- The optimization of the PPO agent : Based on the comparative results, we identify PPO as the top- performing algorithm and further refine it for path tracking tasks. We adjust the neural network architecture and tune hyperparameters (e.g., learning rate, batch size, clipping ratio, entropy regularization, etc.) to improve training stability and policy performance. The optimized PPO agent achieves significantly better tracking accuracy, smoother control inputs, and zero safety violations, outperforming the default configuration on all key metrics.

Overall, our study shows that a combination of multiple-body simulation, deep RL, and formal verification can yield a robust and safe control policy for a free-floating space robot. In the following, we first describe the system modeling and RL framework, then detail the formal safety components, present a comparative analysis of the RL agents, and finally discuss the optimized PPO agent's performance, before concluding with broader insights and future directions.

II. RELATED WORK

A. CLASSICAL CONTROL TECHNIQUES FOR SPACE ROBOTS

[R2.2] Model-based approaches have historically dominated space robot control. Computed torque control and

TABLE 1: [R2.3] Recent advances in Deep RL-based motion management for free-floating space robots. N/R = not reported.

Contribution	RL Alg.	Task Objective	DOF	Simulator	Obs./Act. Dim.	Safety Limits	Eval. Metrics	Formal Ver./ Base Dist. Min.
[9], 2023	SAC	Obstacle avoidance	7	OpenAI Gym	14 / 7D vel.	Reward-based	Success rate	No/No
[10], 2024	DDPG	Collision avoidance	6	Custom	N/R / 6D vel.	Reward-based	Success rate	No/No
[11], 2024	PPO	Joint failure adaptation	7	MATLAB Simscape + SPART	32 / 7D rates	Pos./att. threshold	Per-joint success rate	No/No
[12], 2024	DDPG	Path planning	6	Custom	32 / 6D vel.	Reward-based	Convergence, task completion	No/No
[13], 2025	PPO	Trajectory planning	6	PyBullet	N/R / 6D	Joint limits	Success rate, pos./ori. error	No/No
[14], 2025	LSAC	Trajectory post-processing	N/R	Custom + air-bearing	N/R / N/R	Uncertainty bounds	Tracking error	No/No
[15], 2025	DDPG	Trajectory planning	N/R	Custom	N/R / N/R	Base dist. (soft)	Reward, base dist. red.	No/Yes
Ours	**PPO**	**Cartesian EE tracking**	**4**	**MATLAB/Simulink + Simscape**	**23 / 4D τ**	d_{safe}, $\tau_{\max}$, joint limits	K1–K9	**Yes/Yes**

resolved-motion rate methods leverage accurate dynamic models to achieve precise end-effector positioning [1]. Reactionless path planning explicitly minimizes base disturbance by exploiting the robot's redundancy, while attitude regulation methods use reaction wheels or thrusters to counteract momentum transfer [2], [7]. These approaches offer strong performance guarantees and interpretability when the system model is accurate. However, they are sensitive to model uncertainties and unmodeled dynamics, require significant engineering effort for model identification, and are ill-suited to the data-scarce, uncertainty-rich conditions of on-orbit operations.

B. REINFORCEMENT LEARNING-BASED CONTROL TECHNIQUES

RL has demonstrated success in various robotic manipulation tasks [22], [30]. Policy gradient methods (PG, PPO, TRPO) and actor-critic algorithms (DDPG, TD3, SAC) have been widely applied to continuous control problems [17]–[21].

[R2.2] Applied to free-floating space robots, RL-based methods have been explored for path planning [12], [16], obstacle avoidance [9], trajectory tracking [13], and failure management [11]. The key advantage of RL is its ability to synthesize controllers without explicit dynamic models, making it attractive when system parameters are uncertain or unavailable. Nevertheless, current RL-based approaches share important limitations: most studies evaluate only a single algorithm, omit base attitude minimization as a control objective, and lack systematic benchmarking across comparable conditions.

C. HYBRID SAFETY-AWARE CONTROL SYSTEMS

Safe RL integrates safety constraints through formal verification, barrier functions, and constrained optimization [24], [25], [31], [32].

[R2.2] Shielding approaches [25] intercept unsafe actions at runtime and substitute corrective commands, while control barrier functions (CBFs) [32] encode safety as a continuously enforced constraint on the policy's action set. These hybrid methods combine the adaptability of RL with formal guarantees, ensuring critical properties such as joint limit compliance and collision avoidance are never violated. However, their application to free-floating space robots remains largely unexplored: existing works rarely integrate formal verification directly into the RL training loop or combine it with base attitude minimization objectives.

D. BENCHMARKING STUDIES

[R2.2] Comparative evaluations of RL algorithms for space robot control are scarce. Table 1 surveys recent deep RL contributions to free-floating space robot motion management. As shown, all surveyed works address only individual tasks with a single algorithm, and none simultaneously combines multi-algorithm benchmarking, base disturbance minimization, and formal safety verification. This absence of unified evaluation makes it difficult to draw conclusions about relative algorithm performance and to guide algorithm selection for future missions.

E. RESEARCH GAP

[R2.1] The survey above reveals three compounding gaps in the existing literature. First, no prior work systematically compares multiple RL algorithms under identical conditions for Cartesian end-effector tracking of free-floating space robots. Second, base attitude minimization—a safety-critical requirement in on-orbit operations—is neglected in most RL-based contributions (see Table 1). Third, the integration of formal safety verification directly into the RL training and evaluation loop has not been demonstrated for this class of systems. The present work addresses all three gaps simultaneously: we benchmark six state-of-the-art RL algorithms under unified conditions, incorporate base attitude minimization as an explicit control objective, and embed formal verification of safety constraints throughout training and testing.

III. SYSTEM MODELING AND SIMULATION ENVIRONMENT

A. ADVANTAGES OF AI-ENDOWED DIGITAL TWIN TECHNOLOGY IN ON-ORBIT SERVICING

Recent advances in AI-driven digital twinning have demonstrated considerable benefits for on-orbit servicing, ranging from automated lifecycle management of satellites to the orchestration of skillful agents that autonomously carry out complex assembly, integration, and testing tasks [33], [34]. By combining contextualized real-time sensor data with AI that accommodates uncertainties, digital twins bridge the physical and digital world with accelerated simulation-driven analysis under desired operational conditions and at low costs. Verification and validation also benefit from this paradigm, as changes in system design or operational scenarios can be described in scenario spaces and integrated with digital twins for flexible verification [35]. [R1.1] In this work, the Simulink-based simulation constitutes the digital twin, and the AI component is embodied by the deep reinforcement learning agent—specifically PPO—that is trained and evaluated within it. This distinguishes the approach from classical simulation: rather than relying on pre-specified controllers, the digital twin hosts a learning-based agent that autonomously synthesizes control strategies from interaction, accommodating complex and partially unknown dynamics without requiring explicit model knowledge.

B. ROBOT MODEL

The virtual testbed for this research allows for the simulation of a free-floating space manipulator inspired by the Spacecraft Robotics Toolkit (SPART) example [36]. The robot consists of a cuboid base (serving as the spacecraft) with a 4-degree-of-freedom (4-DOF) serial manipulator arm mounted on it (see Figure 1). The arm has four revolute joints and four links, and its kinematic structure and inertial properties are defined in a Unified Robot Description Format (URDF) file (SpaceRobot.urdf) which is imported into Simulink's multibody simulation environment [26]–[28]. Identified values (see, .e.g., [37]) were chosen for the geometry and the inertial properties of links of the space arm considered in this work and stored in the URDF model. The result is a realistic multibody model with a floating base and an arm, capturing the strong dynamic coupling between them [2].

C. DYNAMIC SIMULATION IN MATLAB/SIMULINK

The simulation is implemented in Simulink using the Simscape Multibody physics engine. Gravity is turned off (set to $[0\ 0\ 0]^T$) to emulate the microgravity orbital environment. The base is connected to the inertial world frame through an unactuated 6-DoF joint [38], allowing it to move freely translate and rotate. No external forces or torques act on the system. Thus, the base only moves in response to forces generated by the manipulator's joints. This configuration ensures that momentum is conserved, motions of the arm can lead to a reaction of the base [2], [7], [29], [32], [38].

[R3.2] The simulation operates under a set of idealizing assumptions that are standard in RL-based space robotics research and are explicitly acknowledged here to facilitate reproducibility and fair comparison. First, ideal sensing is assumed: all state variables (joint angles, joint velocities, base orientation quaternion, and base angular velocity) are read directly from Simulink signal buses without sensor noise, quantization errors, or measurement bias. Second, zero communication latency is assumed: the RL agent receives observations and issues torque commands within the same discrete time step, with no transmission delay between sensing and actuation. Third, the robot is modeled as a rigid multibody system: flexible link dynamics, joint compliance, and thermal drift are not included. Fourth, no external disturbances act on the spacecraft: gravity gradients, atmospheric drag, solar radiation pressure, and residual magnetic torques are all set to zero, consistent with the free-floating assumption in low-Earth orbit. These assumptions establish a clean, controlled benchmark environment in which performance differences can be attributed unambiguously to algorithmic choices rather than to environmental variability. Their impact on real-world deployment is discussed in the Limitations section.

The Simulink model is composed of subsystems, as illustrated in Figure 2. The Robot Plant subsystem computes the full 6-DOF multibody dynamics of the free-floating robot at each time step (see Figure 3), taking joint torque inputs and outputting the resulting robot states by numerically integrating the equations of motion [2]. The Controller subsystem executes the learned RL policy, applying joint torque commands that pass through saturation and safety verification blocks before reaching the plant. If the collision monitor signals an imminent collision, this block issues an emergency stop and gradually overrides all torques to zero [24], [25]. The Reference Trajectory Generator provides the desired end-effector trajectory and base attitude (see Figure 4), outputting the current target position and velocity at each time step. Various sensors measure the robot's state (joint angles, velocities, base orientation, base angular velocity), combining them into an observation signal for the RL agent (detailed in section IV). Finally, safety monitors enforce constraints. For instance, a Collision Monitor checks distances between robot components and triggers a safety halt if any distance falls below a threshold d_{safe}, while a joint-limit monitor triggers an emergency stop if any joint exceeds its allowed range [24], [25], [39].

To accurately simulate the continuous dynamics while interfacing with the discrete-time RL agent, a consistent integration scheme is used. The Simscape solver is configured to operate with a fixed-time step equal to the agent's decision interval (Δt), instead of the default variable-step, to ensure

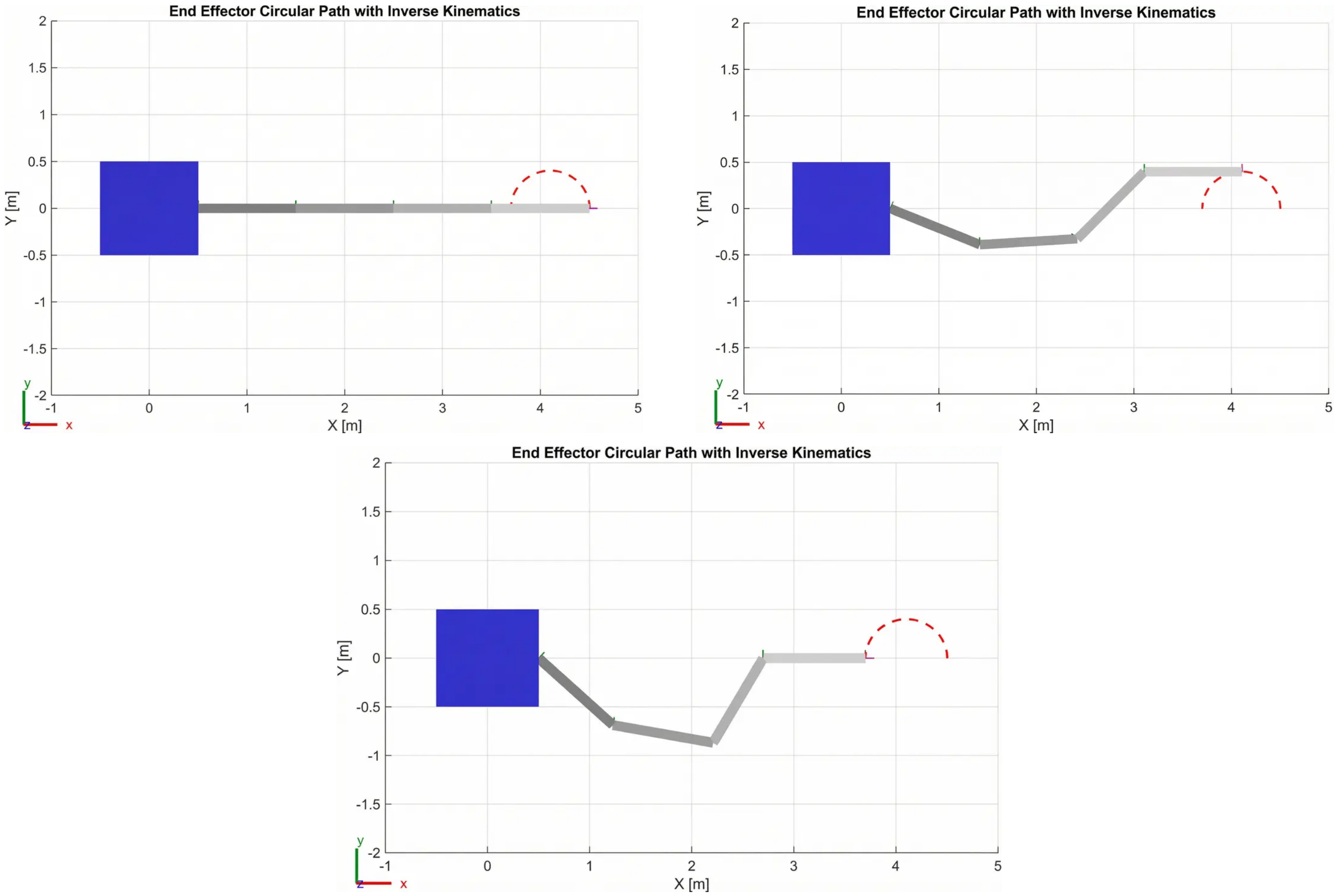


FIGURE 1: Tracking the desired end-effector circular path using inverse kinematics.

synchronization between the physics simulation and the RL decision-making cycle. Rate transition blocks handle any rate differences, avoiding aliasing and ensuring that state observations and action commands align appropriately between the continuous plant and the discrete agent loop [40], [41]. Additionally, Simulink's unit consistency checks are enabled to catch any modeling errors (e.g. mismatched units) early in development.

D. REFERENCE TRAJECTORY AND DESIRED TASK

The target task for the robot is to follow a periodic end-effector trajectory while minimizing the base disturbance. As previously mentioned, the reference trajectory is chosen as a circular path (see Figure 1). Prior to training the RL agents, the trajectory is validated through inverse kinematics to ensure it is geometrically feasible for the manipulator (within joint limits and avoiding singularities). The inverse kinematics solution produced a consistent set of joint movements that achieve the desired end-effector motion, confirming that the circle can be tracked by the arm without requiring unrealistic motions from a kinematic viewpoint. This precaution guarantees that the task is achievable, at least by some controller, so if the RL agent fails to learn it, it is due to learning limitations rather than an unattainable desired Cartesian poses of the end-effector. During simulation, the reference subsystem outputs the desired end-effector position and velocity as time series $EE_{ref}(t)$ and $EE_{vref}(t)$ in the workspace at each time step. From these terms, the instantaneous tracking error signals are computed for the position and velocity of the end-effector. These error signals are key components of the observation and are also used in the reward computation that penalizes large errors.

IV. REINFORCEMENT LEARNING FRAMEWORK

State (Observation) Space: The RL agent receives an observation capturing the system's key state information relevant to control at each time step [22]. The state vector is composed of the following elements:

- End-Effector Tracking Error: The position error $e_p \in \mathbb{R}^3$ between the end-effector's current position and the reference target position (in Cartesian coordinates). If the end-effector is exactly on the desired trajectory point, $e_p = 0$. Additionally, the error in end-effector velocity $e_v \in \mathbb{R}^3$ (difference between current and target end-effector velocities) is included. Together, (e_p, e_v) provides the agent a direct indication of tracking performance. Specifically, it tells the agent how far off the trajectory the end-effector is and whether it is lagging or

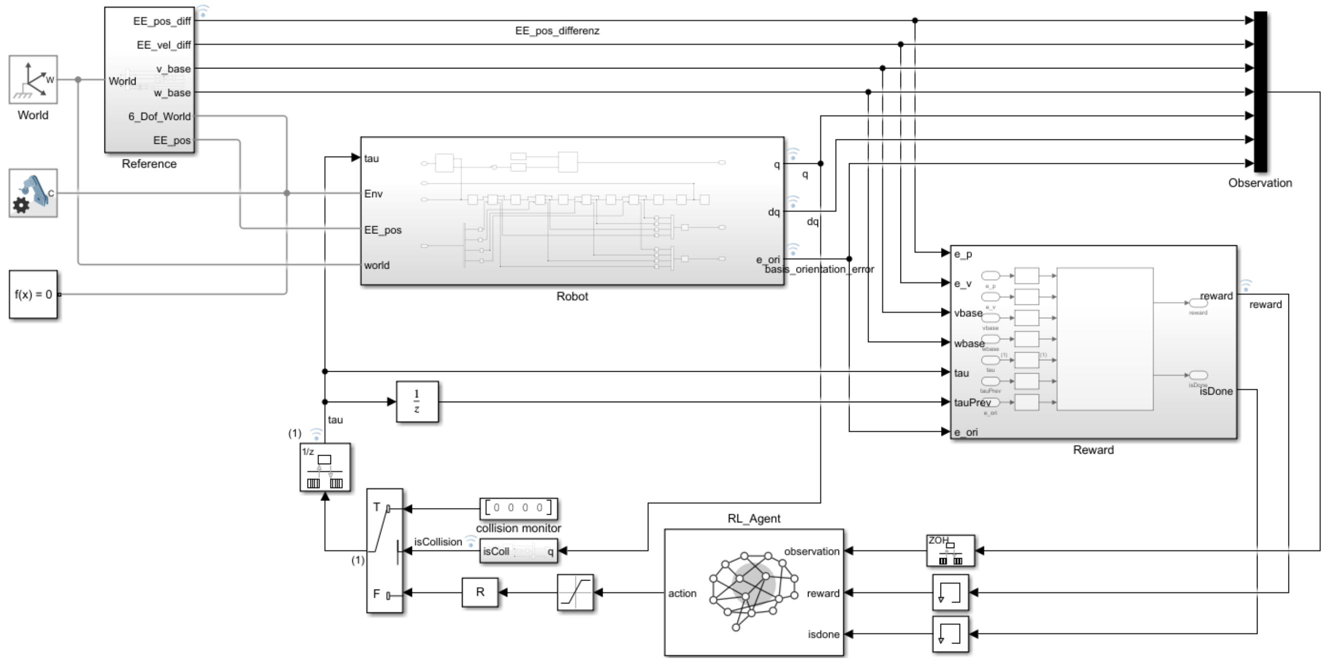


FIGURE 2: Overview of the Simulink model `SpaceRobot.slx`
This figure shows the Simulink-based closed-loop simulation framework for the free-floating space robot. The model integrates the robot dynamics, reference generation, observation assembly, reward computation, and a PPO-based reinforcement learning agent that outputs joint torques. The reward and termination logic evaluate end-effector tracking performance, base motion disturbance, and control effort to guide policy learning.

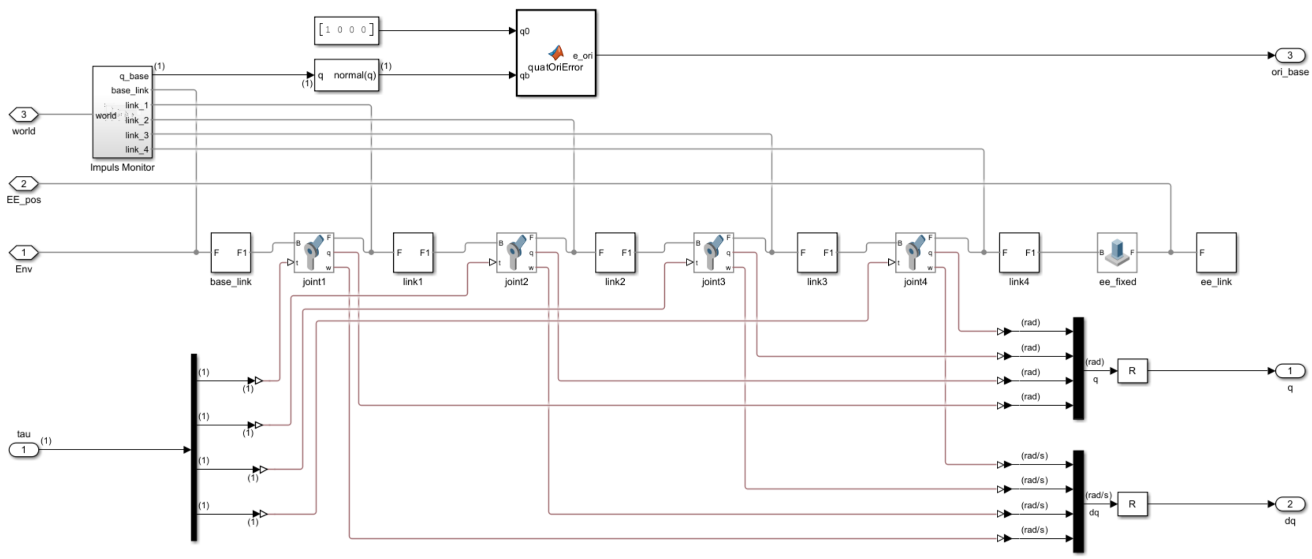


FIGURE 3: Overview of the Robot Subsystem in the Simulink Model `SpaceRobot.slx`
This figure depicts the Simulink/Simscape Multibody model of the free-floating space robot, consisting of a floating base and a serial 4-DOF manipulator. Joint torque commands are applied to the revolute joints, while joint positions and velocities are measured and fed back as state variables. The base orientation is represented as a normalized quaternion and compared to a reference to compute the base attitude error.

overshooting in velocity.

- Arm Joint States: The positions and velocities of each of the 4 arm joints. We include the joint angles $q \in \mathbb{R}^4$ and joint angular velocities $\dot{q} \in \mathbb{R}^4$ in the observation. These inform the agent about the manipulator's configuration and motion state (e.g., at rest or not). (Note: Joint

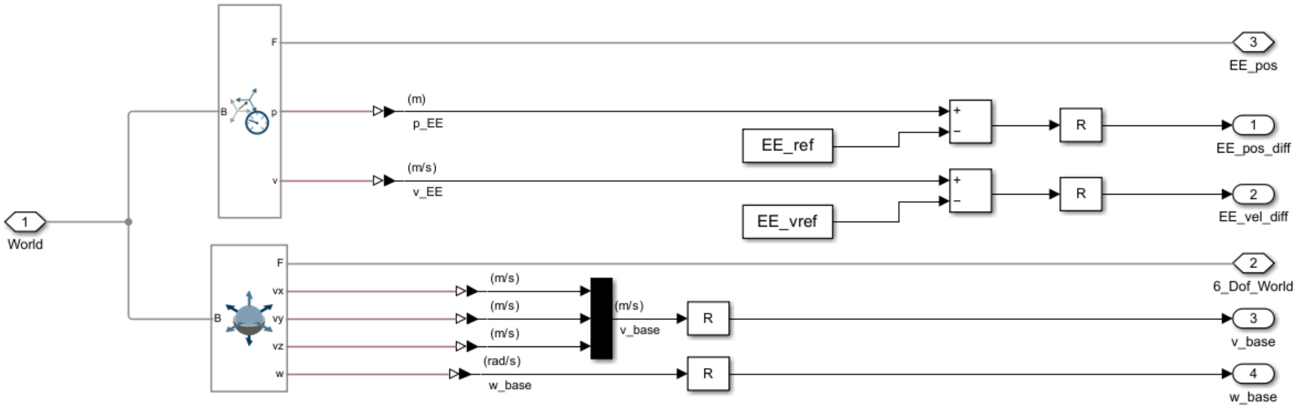


FIGURE 4: Overview of the Reference Subsystem in the Simulink Model `SpaceRobot.slx`
This figure shows the computation of end-effector position and velocity tracking errors with respect to the reference trajectory. The measured end-effector states are compared to the reference signals to generate position and velocity error vectors used for observation and reward calculation. In addition, the base linear and angular velocities are extracted to capture base motion induced by manipulator actuation.

angles can be normalized or wrapped as needed, in our implementation they are provided in radians along with known joint limits.)

- Base Motion: The free-floating base's orientation and angular velocity. We parameterize the base's orientation using quaternions. The quaternion $q_{base} \in \mathbb{R}^4$ describes the orientation of the base, capturing its attitude error from a reference orientation. Since the base is expected to maintain its initial attitude, the orientation error is ideally kept near zero through the agent's actions. The base's angular velocity $\omega_{base} \in \mathbb{R}^3$ is also included, as it captures any ongoing rotation of the base.

Combining these components, the overall observation vector has dimension $3(e_p) + 3(e_v) + 4(q) + 4(\dot{q}) + 3(ori_{base}) + 3(\omega_{base}) + 3(v_{base}) = 23$ dimensions. This rich state description provides the agent with all necessary information: how far the end-effector is from target, how the arm is configured/moving, and how the base is reacting.

Action Space: The agent's action is a vector of continuous joint torque commands for the manipulator's joints. At each decision step, the agent outputs $a = [\tau_1\ \tau_2\ \tau_3\ \tau_4]^T$, corresponding to the torque to apply at each of the 4 revolute joints of the arm [22], [30]. These actions are real-valued and are constrained by the actuator capabilities. We enforce symmetric torque limits: each $|\tau_i| \leq \tau_{\max}$ (for i = 1..4). In the simulation, this is implemented using saturation blocks that clip the commanded torque to the range $[-\tau_{\max}, +\tau_{\max}]$. The value of $\tau_{\max}$ is set based on the robot's design (for example, $\tau_{\max} = 25Nm$ for all joints, though the exact value is not critical as long as it is sufficient to follow the trajectory). These saturation limits not only model physical actuator limits but also serve as a safety measure to prevent excessive torques. The action space is thus continuous $\mathbb{R}^4$, making this a continuous control problem that suits policy gradient and actor-critic RL methods.

The agent's policy (the controller we are learning) is a mapping from the 23-dimensional state space to the 4-dimensional action space: $\pi : s \mapsto a$. Internally, this policy is represented by a neural network whose parameters are adjusted during training to maximize cumulative reward. The control frequency (agent decision frequency) is chosen such that the agent outputs a new torque command at a fixed interval Δt (e.g., 0.1 s). This Δt is the "agent sampling time" in Simulink and is consistent with the simulation's fixed step to maintain alignment.

Reward Function Design: A crucial aspect of the reinforcement learning formulation is the reward signal, which guides the agent toward accurate trajectory tracking while minimizing base disturbance and enforcing smooth, safe control actions. At each discrete time step t, the reward r_t is computed as the sum of a progress term and a proximity bonus, minus a weighted cost function:

$$r_t = \frac{1}{C}\left(r_{\text{prog}} + r_{\text{bonus}} - c_t\right), \tag{1}$$

where C is a normalization constant.

To encourage monotonic convergence toward the reference trajectory, a progress reward is defined based on the reduction of the end-effector position error:

$$r_{\text{prog}} = \begin{cases} k_p\left(d_{t-1} - d_t\right), & d_{t-1} > d_t, \\ 0, & \text{otherwise}, \end{cases} \tag{2}$$

where $d_t = \|e_p(t)\|$ is the Euclidean end-effector position error and k_p is a scaling factor. The previous distance d_{t-1} is stored persistently across time steps.

A small shaping bonus is added when the end-effector is close to the reference point:

$$r_{\text{bonus}} = k_b \exp\left(-\left(\frac{d_t}{\sigma}\right)^2\right), \tag{3}$$

which improves local convergence behavior near the desired trajectory.

The instantaneous cost c_t penalizes tracking errors, base motion, and control effort:

$$c_t = W_p\|e_p\|^2 + W_v\|e_v\|^2 + W_{\text{ori}}\|e_{\text{ori}}\|^2 + W_{\omega_b}\|\omega_{\text{base}}\|^2 + W_{v_b}\|v_{\text{base}}\|^2 + W_u\|\tau\|^2 + W_d\|\tau - \tau_{t-1}\|^2, \quad (4)$$

where e_p and e_v denote the end-effector position and velocity errors, e_{ori} the base orientation error, v_{base} and ω_{base} the linear and angular base velocities, and τ the joint torque vector. The final term penalizes changes in control input to promote smooth actuation.

The weights used in all experiments are summarized in Table 2.

[R2.4] The physical intuition behind the weight choices is as follows. The end-effector position error weight $W_p = 150$ is large because trajectory tracking is the primary task objective; a high penalty ensures the agent prioritizes reducing the position error above all other considerations. The velocity error weight $W_v = 25$ is lower than W_p to avoid over-penalizing transient dynamics during fast arm motion, while still discouraging persistent velocity offsets. The base orientation weight $W_{\text{ori}} = 200$ is the largest in the cost function, reflecting that base attitude disturbance is safety-critical in on-orbit operations: any uncontrolled rotation of the spacecraft base can jeopardize mission success independently of tracking performance. The base angular velocity weight $W_{\omega_b} = 8$ provides a damping-like penalty on ongoing base rotation, while the linear velocity weight $W_{v_b} = 2$ is kept small because, in a free-floating system with zero initial linear momentum, base translational motion remains negligible by design. The torque magnitude weight $W_u = 0.02$ and torque-rate weight $W_d = 0.06$ are intentionally small to leave the agent sufficient freedom to explore diverse control actions; the torque-rate penalty is slightly larger to specifically discourage high-frequency chattering in the control signal. The progress scaling factor $k_p = 10$ is chosen so that the progress reward is of comparable magnitude to the cost terms at typical error levels, preventing it from being dominated. The proximity bonus parameters $k_b = 0.05$ and $\sigma = 0.02$ are small by design: they provide a weak local guidance signal near the trajectory without creating a strong local attractor that could trap the policy. Finally, the normalization constant $C = 500$ scales the total reward to a range of approximately $[-1, 0]$ under typical operating conditions, which stabilizes value function learning across all tested algorithms.

[R2.7] At each step, all inputs to the reward function are checked for numerical validity. If any NaN or Inf values are detected, or if the end-effector error exceeds a predefined bound, the episode is immediately terminated and a terminal penalty of $r_{\text{fail}} = -1$ is assigned. The choice of this value is deliberate and calibrated to the reward scale: as noted above, the normalization constant $C = 500$ compresses step rewards to approximately $[-1, 0]$ under normal operating conditions, meaning $r_{\text{fail}} = -1$ equals the lower bound of a single step reward. This makes the terminal penalty equivalent to receiving a maximally bad step — large enough to strongly discourage unsafe exploration, yet not so extreme as to dominate the cumulative return over an entire episode or destabilize gradient estimates. A much smaller penalty (e.g., -0.1) would be insufficient to deter the agent from entering invalid states, while a much larger penalty (e.g., -100) would cause the failure signal to overshadow all other reward information, potentially leading to overly conservative policies or value function divergence. By anchoring the failure penalty to the step-reward scale, the policy learns a balanced trade-off: failures are clearly undesirable, but the agent is not driven to paralysis by catastrophic gradient magnitudes.

All reward components were implemented in a MATLAB Function block within the Simulink environment.

Training Procedure: We adopt an episodic training approach. Each episode corresponds to a simulation of fixed duration. A finite horizon of H seconds or a fixed number of time steps N. In our case, one episode lasts long enough for the end-effector to attempt following the circular trajectory (e.g., one or two loops of the circle). At the start of each episode, the system is reset to an initial condition by a custom reset function. The initial joint angles are set to a feasible configuration (e.g., the arm in a certain pose near the start of the trajectory) and the base is at a defined attitude (often the home orientation and at rest). All state variables, observer integrators, and logging signals are reset. We ensure the initial state is within safe bounds (no collisions or limit violations at t=0). The reset function can randomize the starting joint angles within a range or the phase of the trajectory to improve generalization, though within this study, we kept a consistent start pose for simplicity (with the end-effector at a specific point on the circle) [22].

Once an episode begins, the agent receives observations at each step and outputs torque actions. The simulation runs until either the time horizon is reached (the episode length H) or a terminal condition occurs (task completed or safety violation). As described, reaching the end of the trajectory

TABLE 2: Reward function weights used in all experiments.

Term	Symbol	Value
End-effector position error	W_p	150
End-effector velocity error	W_v	25
Base orientation error	W_{ori}	200
Base angular velocity	W_{ω_b}	8
Base linear velocity	W_{v_b}	2
Joint torque magnitude	W_u	0.02
Torque rate (smoothness)	W_d	0.06
Progress scaling factor	k_p	10
Proximity bonus scaling	k_b	0.05
Proximity width	σ	0.02
Reward normalization constant	C	500

successfully or experiencing a failure condition will terminate the episode prematurely. In practice, an episode might end early if, for example, the agent causes a collision at 70% of the trajectory, it will receive the penalty and the episode stops there.

We use a standard policy gradient training loop (as provided by MATLAB's RL toolbox): after each episode (or after a set number of steps), the collected state, action, reward trajectories are used to update the agent's policy (and value function, if applicable) via the chosen RL algorithm [42]. The particularities of each algorithm (PPO, DDPG, etc.) are handled by their respective implementation, we supply the experience and let the algorithm update the neural network [22]. All agents were trained for the same number of episodes to allow fair comparison. We chose $N_{train} = 1000$ episodes for each agent, which was sufficient for most algorithms to converge or reach a performance plateau. During training, we monitored metrics such as the cumulative reward per episode, the end-effector tracking error, and the base motion, averaged over each episode. A successful training run is characterized by the episode reward increasing (toward 0, since negative costs are used) and the tracking error and base movement decreasing over time. We also tracked the number of episodes that ended prematurely due to safety triggers, aiming for this to go to zero as training progresses.

[R2.8] To ensure reproducibility, all training runs were initialized using MATLAB's Mersenne Twister random number generator with a fixed seed of zero (`rng(0,'twister')`), applied prior to each training session. For each of the six algorithms, **three independent training runs** were performed under identical conditions—same environment, reward function, episode length, and initial state—resulting in 18 training runs in total. The policy selection procedure was as follows: after all three runs per algorithm completed, the policy checkpoint achieving the highest mean episodic reward over the final 50 training episodes was identified. This checkpoint was then used exclusively for all subsequent evaluation experiments. We found relatively consistent results across runs for the more stable algorithms (PPO, TRPO), whereas some runs of less stable methods (e.g. DDPG) would diverge, underscoring the importance of algorithmic stability in this setting.

Evaluation and Metrics: After training, we evaluated each agent's performance on $N_{eval} = 30$ independent episodes from a fixed initial configuration to gather statistics. Over these evaluation runs, we compute a set of Key Performance Indicators (KPIs) to quantitatively compare the agents.

[R3.10] The nine KPIs are organized into three groups to aid interpretation: tracking accuracy (K2, K3 measure how closely the end-effector follows the reference path), base stability and safety (K4 measures spacecraft attitude drift; K5 and K6 capture constraint violations), and efficiency (K7 quantifies control smoothness; K8 measures residual base rotation; K9 measures energy consumption). K1 serves as an overall scalar summary of the agent's performance across all objectives, since it is the cumulative reward that jointly penalizes all undesired behaviors.

- **K1** — Average Episodic Return: mean cumulative reward per episode; higher is better and reflects overall task success.

$$R_i = \sum_{k=1}^{T_i} r_i[k], \qquad K1 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} R_i.$$

- **K2** — Mean Squared EE Position Error: average squared tracking error per episode; lower is better.

$$K2 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|e_{p,i}[k]\|_2^2.$$

- **K3** — Maximum EE Position Error: worst-case tracking deviation; lower is better.

$$K3 = \max_i \max_k \|e_{p,i}[k]\|_2.$$

- **K4** — Mean Base Orientation Error: average attitude drift of the spacecraft base (quaternion error magnitude); lower is better.

$$K4 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|e_{R,i}[k]\|_2.$$

- **K5** — Collision Rate: fraction of episodes with at least one collision; zero is ideal.

$$C_i = \mathbb{I}(\exists k:\ c_i[k] > 0.5), \qquad K5 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} C_i.$$

- **K6** — Joint Limit Violation Rate: fraction of time steps across episodes where any joint exceeds its mechanical limit; zero is ideal.

$$K6 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i n_J} \sum_{k=1}^{T_i} \sum_{j=1}^{n_J} \mathbb{I}\big(q_{i,j}[k] \notin [q_j^{\min}, q_j^{\max}]\big).$$

- **K7** — Control Smoothness (Jerk): second difference of joint torques; lower values indicate smoother, less chattering actuation.

$$\Delta^2 \tau_i[k] = \tau_i[k+2] - 2\tau_i[k+1] + \tau_i[k], \qquad K7 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i - 2}$$

- **K8** — Mean Base Angular Velocity: residual spacecraft rotation rate during the task (rad/s); lower is better, ideally zero.

$$K8 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|\omega_{b,i}[k]\|_2.$$

- **K9** — Energy Consumption: average absolute mechanical power integrated over each episode; lower is better when not at the expense of tracking quality.

$$E_i = \int \sum_{j=1}^{n_J} \left|\tau_{i,j}(t)\,\dot{q}_{i,j}(t)\right| dt, \qquad K9 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} E_i.$$

[R3.6] Two KPIs deserve explicit justification in the space mission context. **K7 (Control Smoothness)** captures high-frequency torque oscillations (chattering): in space, such oscillations cause accelerated mechanical wear on actuators that cannot be serviced or replaced on orbit, excite structural vibrations that degrade the pointing accuracy of sensitive instruments such as cameras or star trackers, and dissipate energy through rapid back-and-forth motion. Minimizing K7 therefore directly prolongs hardware lifetime and protects mission-critical payloads. **K9 (Energy Consumption)** reflects the mechanical work performed by the joints, approximated as $\int |\tau \cdot \dot{q}|\, dt$. Onboard power is strictly limited by solar panel area and battery capacity; every joule spent on control actuation reduces the budget available for communication, thermal management, and payload operation. Reducing K9 without sacrificing tracking accuracy therefore has direct operational value, extending mission lifetime and lowering the thermal load on electronic components.

Additionally, we consider training-related metrics for comparison: T1: variance of the episodic reward in the later stage of training (indicating training stability), T2: variance of the value function or average reward (another stability metric), and T3: training time required to reach 1000 episodes (reflecting sample efficiency and computational load). These help evaluate how easily an algorithm learns in this environment.

By evaluating all agents on these common metrics, we obtain an objective comparison of their performance and characteristics. Section VI presents and discusses these comparative results.

V. SAFETY MONITORING AND CONSTRAINT ENFORCEMENT

Stability and safety are paramount in space robotics. A controller for autonomous space servicing must meet physical constraints and avoid hazardous behaviors, especially when it builds on learned skills expected to handle uncertainties. In this work, we integrate runtime safety monitors and constraint enforcement mechanisms directly into the RL training and execution environment. These mechanisms serve a dual purpose: they protect the simulation from unsafe states during exploration and, through penalty-driven learning, shape the agent's behavior toward safe operation [22], [24].

Momentum Conservation Monitor: In a free-floating system without external forces, the total linear momentum of the robot (base + arm) should remain constant (and in our case, near zero since initial momentum is zero) [2]. We implemented a Momentum Observer that continuously computes the total linear momentum p_{tot} of the system and checks for its conservation (see Figure 5). This is done by attaching virtual sensors to each body (base and links) to measure their velocities and computing $p_{tot} = \sum_i m_i v_i$. In an ideal simulation, p_{tot} stays roughly zero at all times (small numerical deviations on the order of 10^{-6} Ns may occur due to integration errors). If a significant, systematic change in p_{tot} were detected, it would indicate an unintended external force or a modeling error (such as an unmodeled constraint or bug). During our experiments, p_{tot} remained near 0 (within 10^{-6} Ns) throughout each episode with the learned policy, confirming physical consistency (see Figure 6). This momentum check provides confidence that the simulation adheres to Newton's laws and the agent interacts with a physical correct process. This leverages the significance of the obtained policy. The Momentum Conservation Monitor also serves as a diagnostic tool because any momentum drift would pause training for model inspection.

Collision Monitor and Avoidance: We define a minimum allowable distance between certain parts of the robot (for example, between the end-effector and the base structure). A Collision Monitor subsystem computes distances between predefined pairs of bodies each step (using the geometry from the URDF and Simscape's collision detection capabilities). If any distance falls below the safe threshold d_{safe} (indicating a potential collision or self-collision is imminent), the system triggers a collision event (see Figure 7) [39]. In our simulation, we set, e.g., $d_{safe} = 5$ cm as the minimum clearance. When triggered, the following occurs: (1) a warning flag is raised and logged (for analysis), (2) a done signal is sent to terminate the episode immediately, and (3) all joint torque commands from the agent are overridden to zero to stop the robot motion. This mechanism effectively acts as an emergency brake, preventing the agent from continuing once a collision is about to happen. The collision monitor was tested by deliberately commanding the arm toward the base. It reliably detected the violation of the safety distance and stopped the system (see Figure 8). During training of the RL agents, collisions occurred in early exploratory episodes for some agents, but the monitor caught them and ended those episodes with heavy penalties. As training progressed, all agents learned to avoid collisions altogether, indeed, in the final evaluation all agents achieved episode without any collision ($K5 = 0$), confirming the efficacy of this safety measure.

Joint Limit Enforcement: The robot arm has joint angle limits (e.g., each revolute joint might be limited to ±180° or a smaller range based on mechanical constraints). Violating joint limits could damage a real robot and often indicates an unstable controller. We enforce joint limits at two levels. First, in the Simscape model, we include mechanical joint limits with penalty forces (a stiff barrier that resists further motion beyond the limit). Second, we add an Assertion block that monitors each joint angle. If any joint exceeds its predefined limit, the assertion triggers a violation event. Similarly

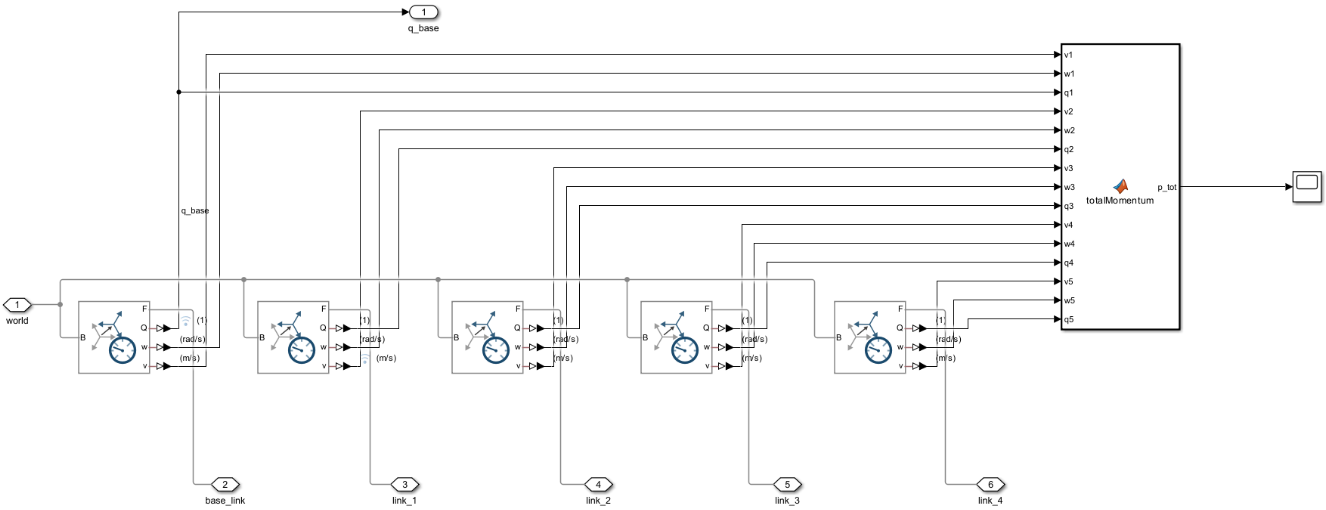


FIGURE 5: Overview of the Impuls Monitor Subsystem (Momentum Observer) in the Simulink Model `SpaceRobot.slx` This figure shows the computation of the total linear and angular momentum of the free-floating space robot. The velocities and orientations of the base and all manipulator links are measured and aggregated within a MATLAB function to obtain the system momentum. This monitor is used to analyze momentum conservation and quantify base–manipulator dynamic coupling effects.

to the collision case, we handle a joint limit violation by: terminating the episode, applying a large negative reward, and zeroing out all torques immediately. In fact, the Simulink model is configured such that when the assertion triggers, it directly sets the torque inputs τ to zero in that same time step as a protective measure. This ensures the robot doesn't push further into the limit or oscillate. In our results, we observed very few limit violations once agents were trained. Among all algorithms, PPO and TRPO had a handful of episodes grazing the limits (K6 values of a few percent), whereas TD3, SAC, PG, and DDPG essentially never hit the limits ($K6 \approx 0$). Notably, PPO's superior tracking performance came at the cost of occasionally using the full joint range (K6 = 0.0505, i.e. ~5% of episodes, had minor limit violations). Thanks to the joint limit enforcement, these events did not lead to instability: the agent's torques were cut off momentarily when a limit was reached, preventing any damaging behavior, and the episode was marked as failed. Over time, the agent learned to avoid this scenario as well,

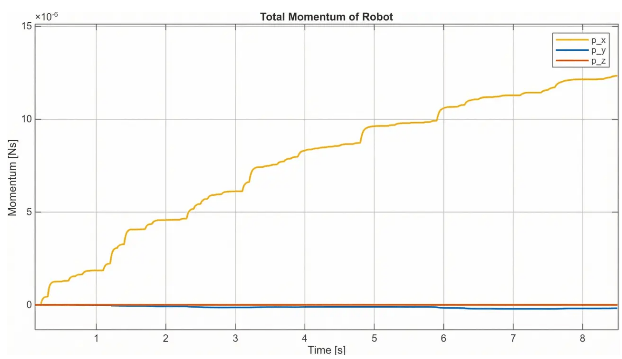


FIGURE 6: Dynamics of the total momentum of the robot.

especially in the optimized PPO version (where K6 dropped to 0).

Formal Verification of Torque Constraints: In addition to enforcing torque saturation during simulation, we performed an offline formal verification to prove that the trained policy could not command out-of-bounds torques. Using MATLAB's Simulink Design Verifier (SLDV) in property-proving mode, we analyzed a simplified closed-loop model of the agent controlling the robot's joints. The property to prove was that for all possible agent inputs (observations), each output torque τ_i remains within $[-\tau_{\max}, +\tau_{\max}]$ (see Figure 9). The verification succeeded: SLDV was able to prove that no counterexample exists and the torque limits are always respected by the policy network. Figure 10 shows the SLDV result confirming the property (no violations found) [23]. This formal assurance is important because it considers all possible states, including ones not seen during simulation, and guarantees the policy cannot output a dangerous torque. It's worth noting that directly running formal analysis on the full Simscape model was not feasible, the nonlinear dynamics are too complex for the automated theorem proving. To circumvent this, we extracted the relevant part of the model (the control law and saturation logic) into a separate simplified model (Verify_tau.slx) where the plant was replaced by a worst-case abstraction (essentially, we just needed to check the policy output block). This exemplifies how formal methods can complement simulation-based learning: by verifying critical properties on an abstract model of the agent, we gain additional confidence in safety beyond empirical testing.

Limitations of Formal Methods: While the above components significantly enhance safety, we observe that they cover a limited set of properties. They ensure zero collisions,

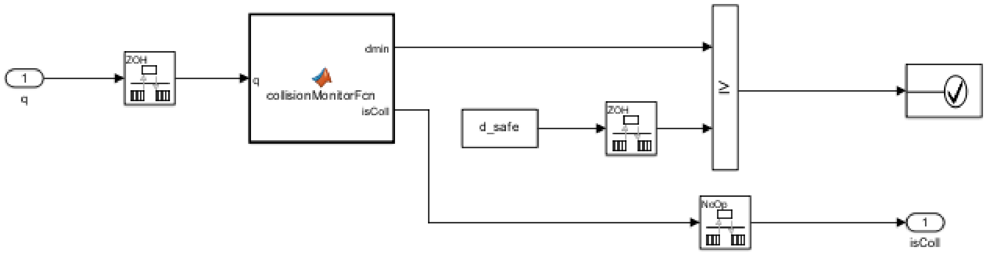
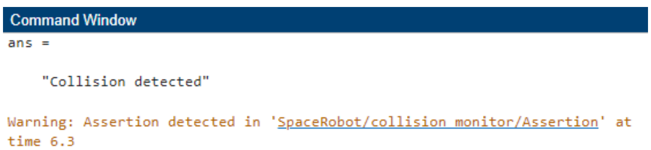
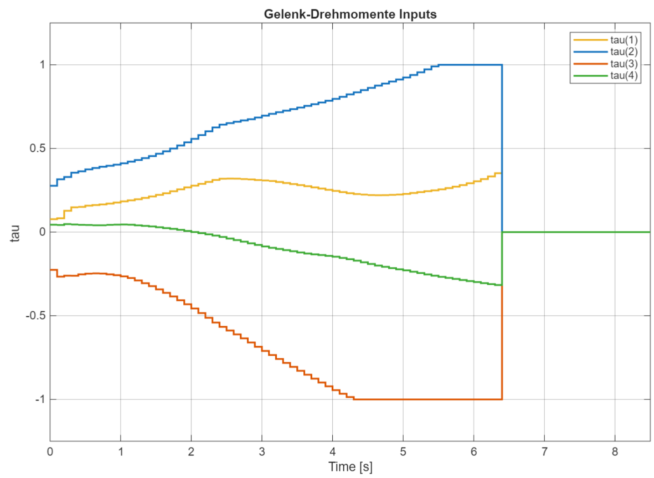


FIGURE 7: Overview of the Collision Monitor Subsystem in the Simulink Model `SpaceRobot.slx`
This figure illustrates the collision monitoring logic used to ensure safe robot operation during training and evaluation. Joint and link configurations are processed to compute the minimum distance to obstacles, which is compared against a predefined safety threshold. A collision flag is generated and used for episode termination or safety-related penalties.



(a) Automatically triggered collision warning



(b) Scope of torque inputs in the event of a collision

FIGURE 8: Representation of collision detection and the associated torque curves.

no joint position overshot, and bounded torques, and they monitor physical consistency (e.g., momentum). However, other aspects, such as guaranteeing task completion within a time or stability margins, were not formally verified. A complete formal specification of the entire mission (for instance, something like temporal logic specifications of "the robot shall accomplish X without Y happening") would be far more complex. Our approach selectively targets the most critical safety concerns. In practice, this proved sufficient to train successful policies without a single catastrophic failure: episodes that would have been catastrophic were safely stopped by the monitors (and heavily penalized to teach the agent). As a result, by the end of training, the learned agent's behavior was entirely meaningful within the safety envelope enforced by the formal constraints [25].

In summary, integrating these formal safety checks into the learning process was vital. It not only protected the simulation (and a potential future real system) from damage during the exploratory phase of learning, but it also shaped the agent's behavior by penalizing unsafe actions. The agent thus learns with a form of built-in "safety shield." This approach demonstrates a viable path to apply deep RL in safety-critical domains like space robotics, where pure trial-and-error without safeguards would be unacceptable. We next analyze the performance outcomes of the various RL agents under these conditions.

[R3.5] **Computational Overhead and Scalability:** The three safety monitors — momentum observer, collision distance checker, and joint-limit assertion — each consist of simple algebraic operations over the robot's link states, scaling as $\mathcal{O}(n_{\text{links}})$ per time step. For the 4-DOF arm considered here, this overhead is negligible compared to the physics simulation itself. End-to-end training times, including the full safety stack, are reported as metric T3 in Table 4: they range from approximately 8 minutes (PG, PPO) to 38 minutes (TD3) for 1000 episodes on a standard workstation (AMD Ryzen 7 7700, 32 GB RAM). For onboard deployment, only the learned policy network needs to execute in the control loop: the two hidden-layer network (128 units per layer, $\approx 17\,000$ parameters) requires only a small number of matrix multiplications and can comfortably run at 10 Hz on modern space-qualified embedded processors (e.g., ARM Cortex-A series). The safety monitors can be retained as lightweight parallel watchdog processes, or reduced to the most safety-critical checks (torque saturation, joint-limit assertion) with minimal additional compute. This modular architecture makes the approach scalable to resource-constrained onboard systems.

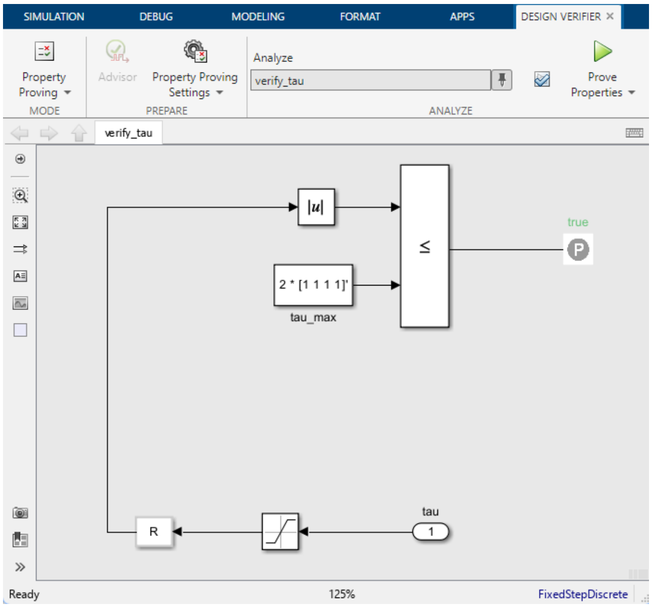


FIGURE 9: Verification model with Simulink Design Verifier `Verify_tau.slx`

This figure depicts the Simulink property model used to formally verify the joint torque constraints. The absolute joint torques are compared against predefined maximum limits using a relational operator. The property is proven to hold for all reachable system states, ensuring bounded control actions.

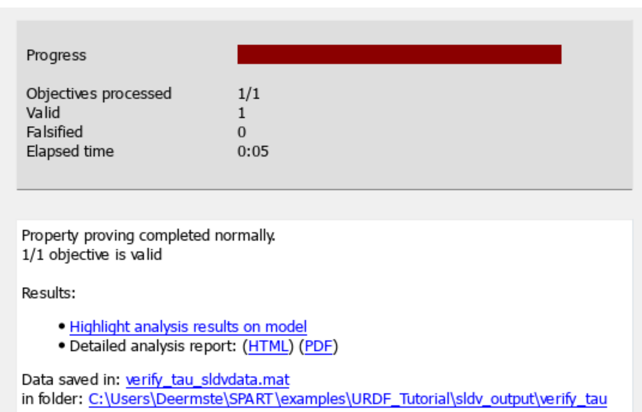


FIGURE 10: Result of the Simulink Design Verifier

This figure shows the result of a formal property verification performed on the torque constraint model. The verification confirms that the specified safety property is satisfied for all analyzed execution paths. This analysis provides formal guarantees on control torque limits beyond empirical simulation-based validation.

VI. COMPARISON OF CONTINUOUS RL AGENTS

A central question of this study is: Which continuous-action RL algorithm is most effective for controlling a free-floating space robot? To answer this, we evaluated six RL algorithms from three major families on the same task and environment [22], [30]. The algorithms compared are:

- Policy Gradient (PG): the classic REINFORCE algorithm (Monte Carlo policy gradient), included as a baseline for direct policy optimization without advanced variance reduction [21], [22].
- Proximal Policy Optimization (PPO): a modern policy-gradient method that uses clipped probability ratio updates to ensure stable and conservative policy improvements. PPO is known for its balance of stability and performance [20].
- Trust Region Policy Optimization (TRPO): a policy-gradient method that enforces a trust-region constraint (via Kullback–Leibler divergence) on policy updates. TRPO is expected to perform similarly to PPO in terms of stability, albeit with higher computational cost due to solving a constrained optimization each update [17].
- Deep Deterministic Policy Gradient (DDPG): an off-policy actor-critic algorithm with a deterministic policy and target networks. DDPG can excel in continuous control by learning a Q-function and a policy simultaneously, but it is known to be sensitive to hyperparameters and can be unstable [43].
- Twin Delayed DDPG (TD3): an improved variant of DDPG that addresses function approximation issues by using two critic networks to mitigate overestimation bias and by delaying policy updates. TD3 generally offers more stable learning than standard DDPG [18].
- Soft Actor-Critic (SAC): an off-policy actor-critic algorithm that learns a stochastic policy and includes an entropy term in the reward to encourage exploration. SAC aims to achieve both high reward and high entropy, making the agent's behavior robust and exploratory. It often achieves very good performance but can be computationally heavier due to sampling and entropy calculations [19].

These algorithms cover a spectrum from on-policy to off-policy, from deterministic to stochastic, and from simpler to more complex update rules. Based on prior knowledge and the nature of our problem, we hypothesized that algorithms with more robust, stable policy updates (like PPO and TRPO) would likely outperform others in this highly coupled, continuous control scenario. Off-policy methods (DDPG, TD3, SAC) might reach good performance as well, but could suffer from tuning sensitivity and stability issues given the complex dynamics and safety constraints. Indeed, DDPG and basic PG were expected to be the least reliable due to known tendencies for instability without additional regularization.

On-policy methods (PG, PPO, TRPO) update their policies using only data from the current policy, leading to stable but less sample-efficient learning. Off-policy methods (DDPG,

TABLE 3: Classification of the evaluated reinforcement learning algorithms.

Algorithm	Policy Type
Policy Gradient (PG)	On-policy
Proximal Policy Optimization (PPO)	On-policy
Trust Region Policy Optimization (TRPO)	On-policy
Deep Deterministic Policy Gradient (DDPG)	Off-policy
Twin Delayed DDPG (TD3)	Off-policy
Soft Actor-Critic (SAC)	Off-policy

TD3, SAC) reuse experience from a replay buffer, improving sample efficiency at the cost of higher sensitivity to hyperparameters [22], [30]. For free-floating space robot control, where strong nonlinear coupling and safety constraints demand reliable convergence, on-policy methods are expected to be advantageous.

Training Setup: All experiments were conducted on a workstation equipped with an AMD Ryzen 7 7700 processor and 32 GB of RAM, using MATLAB R2025b with the Reinforcement Learning Toolbox and Simscape Multibody. We used the same network architecture (initially two hidden layers of 128 units each for policy and for value function, where applicable) and same reward structure for all. Hyperparameters (learning rate, discount factor $\gamma = 0.99$, etc.) were initially set to default reasonable values and kept consistent across agents as much as possible. All agents were trained using a learning rate of $\alpha = 1 \times 10^{-3}$ for both actor and critic networks, unless stated otherwise. Hence, any performance differences can be attributed primarily to the algorithmic differences, not to unequal tuning. Later we will tune PPO further, but for the main comparison, PPO uses a default configuration as well. Each agent's training run of 1000 episodes yielded a final policy which we then evaluated.

Performance Metrics: We evaluated the agents on the KPIs described earlier (tracking error, base stability, etc.) by running 30 test episodes for each. Table 4 summarizes a subset of these metrics, highlighting the differences between agents.

The results in Table 4 reveal clear patterns across the six agents. Regarding training stability, TRPO and PPO exhibited the most stable learning (T1 values of 0.61 and 1.78, respectively), whereas DDPG showed highly erratic behavior (T1 = 64.5). PPO was also the fastest to train (T3 $\approx$ 8 min), while off-policy methods required substantially longer (TD3 $\approx$ 38 min, SAC $\approx$ 35 min).

In terms of trajectory tracking, PPO achieved the lowest mean squared error (K2 = 0.0257), significantly outperforming the runner-up TRPO (0.0680). DDPG achieved the lowest peak error (K3 = 0.4758) in its best runs but was highly inconsistent. PG, SAC, and TD3 all exhibited substantially larger tracking errors, with PG's mean error roughly six times that of PPO.

For base motion minimization, PPO and TRPO both kept the base orientation error small (K4 of 0.0667 and 0.0694), while DDPG allowed the base to drift considerably (K4 = 0.2682). Interestingly, TD3 achieved very low base angular velocity (K8 $\approx$ 0.02 rad/s) by adopting an overly conservative strategy that sacrificed tracking accuracy. All agents successfully avoided collisions (K5 = 0). PPO and TRPO occasionally approached joint limits (K6 of 5.1% and 2.1%, respectively), indicating more aggressive use of the workspace for better tracking.

PPO also produced the smoothest control signals (K7 = 0.6812), while TD3 and DDPG generated the jerkiest outputs ($\approx$ 0.8–1.0). Energy consumption (K9) was moderate for PPO ($\approx$ 6.33) and lowest for TRPO ($\approx$ 3.76); the very low values for DDPG and TD3 (< 1.0) reflect insufficient actuation rather than efficiency. Qualitatively, PPO produced the most competent behavior: the arm moved accurately along the circle with minimal base rotation. TRPO performed similarly but with slightly slower response. SAC tended to oscillate around the path, TD3 adopted an overly conservative strategy sacrificing tracking accuracy, DDPG was inconsistent across runs (explaining its high T1), and PG never achieved reliable tracking within 1000 episodes.

Overall Winner, PPO: Aggregating all the criteria, PPO stands out as the best-performing agent. As summarized succinctly in the report:

- PPO offers the best combination of training stability (low T1/T2), training speed (short T3), tracking accuracy (low K2/K3), and smoothness (low K7), with a moderate joint limit violation rate.
- TRPO is a close second, described as similarly stable and precise but requiring more computation time and still incurring slight limit violations.
- The off-policy methods (TD3, SAC) did not achieve better performance and were significantly more computation-intensive, with higher tracking errors.
- Finally, PG and DDPG were less robust both in training and in achieved performance, showing the largest fluctuations and poorest metrics.

These findings validate our hypothesis that on-policy methods with robust update rules are best suited for this highly coupled, safety-critical control problem. Based on these results, we selected PPO for further optimization.

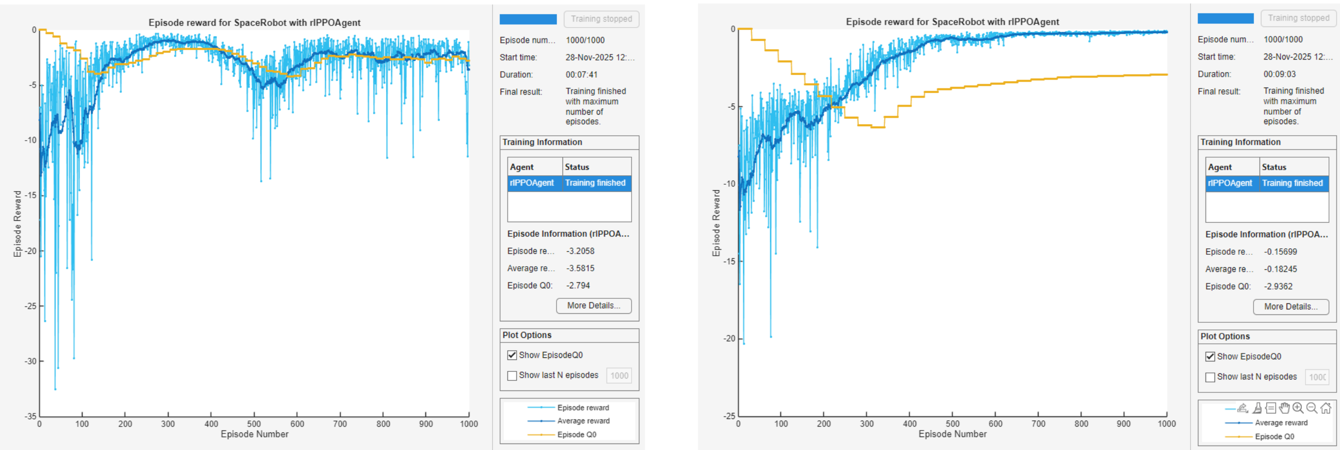
Based on these results, we selected PPO as the reference agent for further development and optimization. The next section discusses how we improved PPO's performance even further through targeted optimizations.

VII. OPTIMIZATION OF THE PPO AGENT

Having identified PPO as the most promising algorithm, we focused on enhancing its performance on path tracking task of the space robot. The PPO agent we compared above was a "default" PPO configuration (as provided by MATLAB's RL Toolbox example) with minimal tuning [42]. While it already outperformed other agents, we hypothesized there was room to improve in terms of faster convergence, higher precision, and eliminating the remaining constraint violations. We therefore undertook a systematic optimization of PPO, involving both network architecture modifications and hyperparameter tuning.

TABLE 4: Selected performance metrics for each RL agent (best values in bold). PPO excels in most categories, achieving the lowest tracking errors (K2, K3), best base stability (K4), and smoothest control (K7), while maintaining short training time. TRPO is a close second in many metrics but requires more training time. Off-policy methods (TD3, SAC, DDPG) show larger errors or instabilities (e.g., high T1 for DDPG) despite some strengths (DDPG's low max error K3). KPI data from evaluation of 30 episodes per agent.

KPI (unit)	TRPO	TD3	SAC	PPO	PG	DDPG
K2: Mean Squared EE position error	0.0680	0.1914	0.1908	**0.0257**	0.1638	0.0937
K3: Max EE error	0.6783	0.5341	0.7766	0.4833	1.0654	**0.4758**
K4: Mean base orientation error	0.0694	0.0868	0.1687	**0.0667**	0.1302	0.2682
K6: Joint limit violation rate	0.0214 (2.1%)	0	0	0.0505 (5.1%)	0	0
K7: Torque smoothness (norm. jerk)	0.7498	0.9981	0.7536	**0.6812**	0.7125	0.8118
T1: Reward std. (late training)	**0.6144**	8.5168	9.6907	1.7756	13.1970	64.4764
T3: Training time (1000 eps)	∼13 min	∼38 min	∼35 min	**∼8 min**	∼8 min	∼24 min



(a) PPO (Default)   (b) PPO (Optimized)

FIGURE 11: Comparison of episodic rewards for default and optimized PPO agents. (a) Episode reward evolution during training using the default PPO hyperparameters. (b) Episode reward evolution using the optimized PPO configuration. The optimized configuration exhibits faster convergence and higher average episode rewards, indicating improved learning efficiency and policy performance.

[R2.10/R3.8] **Network Architecture Improvements:** By default, the PPO agent used two hidden layers of 128 neurons each (with ReLU activations) for both the policy and value function networks. We experimented with deeper and wider networks, as well as alternative activation functions, to better capture the dynamics of the problem. Our optimized architecture ultimately used 3 hidden layers for the policy network with layer sizes [256, 256, 128] and 2 hidden layers for the value network with sizes [256, 128]. We found that the additional depth and capacity in the policy network helped the agent learn the intricate relationship between arm motions and base reactions (especially given the 23-dimensional state input) [42]. We also introduced layer normalization in the first layer to help stabilize learning, given the different scales of input features (joint angles vs. orientation quaternions, etc.). The activation function remained ReLU, which was sufficient. These architecture changes were implemented by explicitly defining the neural network layers instead of using the auto-generated default network. In summary, the optimized policy network was larger and potentially more expressive, which we believe allowed PPO to find a control strategy which is better tuned.

Hyperparameter Tuning: The optimization followed a coordinate-wise grid search: each hyperparameter was swept across a discrete set of candidate values while all others were held at their current best, and the configuration yielding the highest mean episodic reward over the final 50 training episodes (out of $N_{\text{train}} = 1000$) was selected. In total, 16 training runs of 1000 episodes each were conducted during this tuning phase. The swept parameters, their candidate values, and the selected values are as follows:

- **Actor learning rate** — candidates: $\{1 \times 10^{-3},\ 5 \times 10^{-4}\}$; selected: 5×10^{-4}. The lower rate reduced overshooting in policy gradient updates, improving convergence stability.
- **Critic learning rate** — candidates: $\{1 \times 10^{-3},\ 5 \times$

$10^{-4}\}$; selected: $1{\times}10^{-3}$. A higher critic learning rate allowed the value function to track the evolving policy more closely, providing more accurate advantage estimates.

- **Entropy loss weight** — candidates: $\{1\times10^{-2},\ 1\times10^{-3}\}$; selected: 1×10^{-3}. A small entropy bonus discouraged premature convergence to a suboptimal deterministic policy and encouraged the agent to discover strategies such as coordinated joint motions that cancel base momentum, without causing aimless exploration.
- **Experience horizon** — candidates: $\{512,\ 1024,\ 2048\}$; selected: 1024. A moderate rollout horizon provided sufficient trajectory information per update while keeping memory requirements and update latency manageable.
- **Mini-batch size** — candidates: $\{128,\ 256\}$; selected: 256. Larger batches provided more stable gradient estimates per update, reducing training variance.

Through this process we converged on a configuration that provided a more stable and higher-performing PPO agent. Notably, the asymmetric learning rates — a slower actor ($5{\times}10^{-4}$) paired with a faster critic ($1{\times}10^{-3}$) — reflect the different roles of the two networks: the value function must adapt quickly to policy changes, while conservative actor updates prevent destabilizing the learned control strategy.

A. CIRCULAR TRAJECTORY

Training Outcome: The effects of these optimizations were evident in the training learning curves (see Figure 11). The default PPO agent's learning curve initially rose, but then plateaued at a fairly negative reward value (~-2 per episode) and exhibited high variance even after many episodes. This plateau suggested it reached a suboptimal policy and struggled to improve further. In contrast, the optimized PPO agent's learning curve kept rising steadily and converged to a much higher average reward (less negative, closer to zero) with significantly reduced variance. Table 5 reports the KPI averages over $N_{eval} = 30$ evaluation episodes for PPO with the default configuration and the optimized configuration. Specifically, where the default PPO leveled off around -2.0 cumulative reward, the optimized PPO approached -0.4 on average, a dramatic improvement. The variability (shaded region in the reward plot) shrank, indicating the agent's behavior became consistently optimal across training episodes. Quantitatively, the mean episodic reward K1 improved from -2.23 (default PPO) to -0.43 (optimized PPO). This roughly fivefold increase in reward corresponds to the agent accomplishing much more of the task (since zero would be perfect tracking with no penalties). The lower variance also hints that the agent not only improved its average performance but did so reliably without frequent failures.

Beyond the reward improvement, the optimizations translate directly to task performance. Figure 12 shows the end-effector path in the XY-plane: the default PPO deviates significantly from the reference circle and exhibits jagged oscillations, whereas the optimized PPO traces the reference almost exactly. The mean squared tracking error K2 dropped from 0.0257 to 0.0083 (a 67.7% reduction), and the maximum error K3 decreased from 0.4833 to 0.27.

Base stabilization improved markedly (Figure 13): the mean orientation error K4 fell from 0.0667 to 0.022 (factor ≈ 3), and the mean angular velocity K8 was halved from 0.095 to 0.050 rad/s. The optimized agent learned arm motions that effectively cancel momentum transfer to the base.

The optimized PPO also eliminated all safety violations (K5 = K6 = 0 over 30 episodes, compared to 5.1% joint-limit violations for the default), improved torque smoothness (K7: 0.6812→0.351), and reduced energy consumption by 16% (K9: 6.325→5.314). Notably, better tracking, base stability, safety, and energy efficiency were all achieved in parallel, indicating a fundamentally more efficient control strategy.

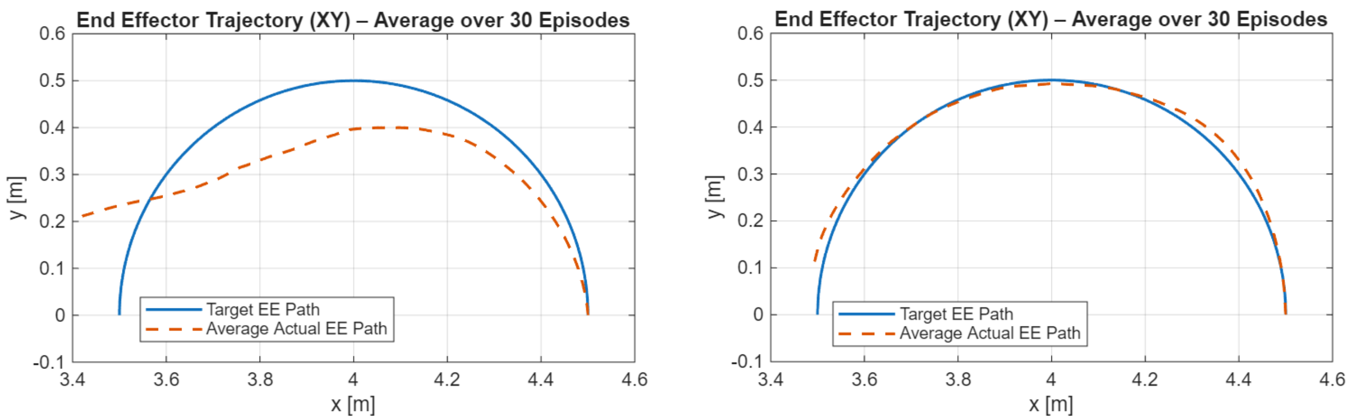
The optimized PPO agent fulfills all task objectives simultaneously: precise trajectory tracking, minimal base disturbance, zero safety violations, smooth actuation, and reduced energy consumption. This validates the approach of systematic hyperparameter tuning and network tailoring for safety-critical space robot control.

B. ABLATION STUDY OF REWARD COMPONENTS

[R2.6] To quantify the contribution of each individual term in the composite reward function defined in Equation 1, we performed an ablation study in which the optimized PPO agent was retrained on the circular trajectory with one reward component disabled at a time. Each of the seven weights in the cost term c_t was set to zero in turn while keeping the remaining six at their baseline values ($W_p = 150$, $W_v = 25$, $W_{\text{ori}} = 200$, $W_{wb} = 8$, $W_{vb} = 2$, $W_u = 0.02$, $W_d = 0.06$). In addition, we separately disabled the two positive reward terms by setting $r_{\text{prog}} = 0$ and $r_{\text{bonus}} = 0$, respectively, to isolate the contribution of the progress signal and the proximity bonus. All other aspects of the training and evaluation pipeline (hyperparameters, random seeds, episode horizon, and the

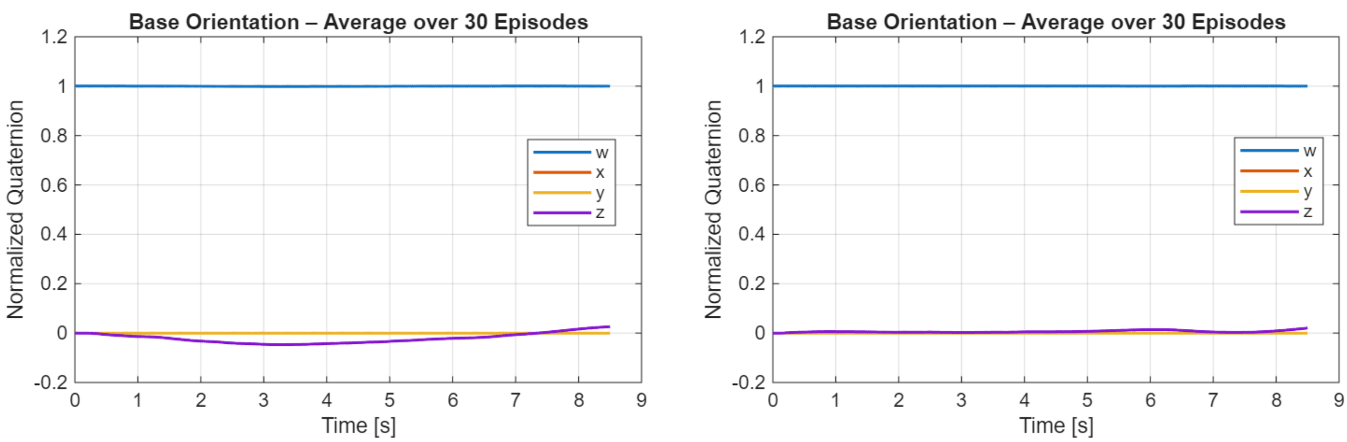
TABLE 5: Performance of PPO agent before and after optimization. The optimized PPO shows substantial improvements in all aspects (higher rewards, lower errors, zero safety violations, smoother and more efficient control).

Metric (KPI)	PPO (Default)	PPO (Optimized)
Mean Episodic Reward (K1)	−2.23	**−0.43**
Mean Squared EE Tracking Error (K2)	0.0257	**0.0083**
Max EE Tracking Error (K3)	0.4833	**0.27**
Mean Base Orientation Error (K4)	0.0667	**0.022**
Collision Rate (K5)	0	0
Joint Limit Violation Rate (K6)	0.0505 (5.1%)	**0**
Torque Smoothness (K7, jerk)	0.6812	**0.351**
Energy Consumption (K9)	6.325	**5.314**



(a) PPO (Default) (b) PPO (Optimized)

FIGURE 12: Target/actual comparison of the end-effector trajectory in the XY plane.



(a) PPO (Default) (b) PPO (Optimized)

FIGURE 13: Course of base orientation (quaternion components) during a test episode.

$N_{eval} = 30$ evaluation protocol) were held identical to the baseline configuration. The resulting KPIs are summarized in Table 6.

Dominant weights (W_{ori}, W_p). Removing the base-orientation penalty is catastrophic: the mean episodic reward K1 collapses from -0.43 to -23.65, and the mean base-orientation error K4 rises by more than an order of magnitude (from 0.022 to 0.668). Interestingly, the end-effector tracking error K2 even decreases slightly, which reveals that without W_{ori} the agent optimizes EE tracking at the expense of completely losing base stabilization. Because base stability is the primary challenge in free-floating manipulation, this confirms that W_{ori} is structurally essential rather than merely fine-tuning the behavior. Removing W_p degrades EE tracking severely (K2 rises by a factor of ≈ 29 and K3 increases approximately $3.5\times$, not shown in the table), driving K1 to -6.48.
Secondary weights (W_v, W_{wb}, W_{vb}). These terms individually have a small but measurable impact: K1 lies in the

TABLE 6: [R2.6] Ablation study of the reward components. Each row corresponds to a separately retrained optimized PPO agent with one reward weight set to zero. KPIs are averaged over $N_{eval} = 30$ evaluation episodes on the circular trajectory.

Variant	K1	K2	K4	K6	K7	K9
Baseline	$\mathbf{-0.43}$	$\mathbf{0.0083}$	$\mathbf{0.022}$	$\mathbf{0}$	$\mathbf{0.351}$	5.314
$W_p = 0$	-6.48	0.238	0.046	0	0.626	2.868
$W_v = 0$	-0.85	0.022	0.049	0.0015	0.655	3.561
$W_{\text{ori}} = 0$	$\mathbf{-23.65}$	0.017	$\mathbf{0.668}$	0.047	0.588	4.363
$W_{wb} = 0$	-1.17	0.032	0.059	0.0022	0.657	3.554
$W_{vb} = 0$	-1.31	0.039	0.054	0.0004	0.670	3.192
$W_u = 0$	-1.32	0.037	0.060	0	0.618	3.856
$W_d = 0$	-1.10	0.029	0.057	0.0003	0.632	3.724
$r_{\text{prog}} = 0$	-3.65	0.075	0.049	0.0173	0.618	3.365
$r_{\text{bonus}} = 0$	-3.96	0.076	0.057	3.8e-5	0.627	4.078

range $[-1.31, -0.85]$ compared to the baseline -0.43, and tracking accuracy and smoothness are noticeably worse (K2 rises by a factor of 2–5, K7 roughly doubles). They shape the motion quality and the base velocity damping, but are not individually load-bearing for task completion.

Control-effort weights (W_u, W_d). Disabling the penalties on torque magnitude and torque change modestly increases tracking error (K2) and roughly doubles the jerk metric K7, while K1 remains around -1.3. This confirms that these terms primarily shape actuation smoothness rather than task completion.

Positive reward terms (r_{prog}, r_{bonus}). Removing either of the two positive reward terms causes a pronounced performance drop, with K1 falling to -3.65 and -3.96, respectively, and the tracking error K2 increasing by roughly an order of magnitude (from 0.0083 to $\approx\ 0.075$). Without r_{prog}, the agent loses the dense directional signal that rewards incremental reduction of the position error, and the joint-limit violation rate K6 rises to 1.73% — indicating that the agent tends to execute more aggressive motions when deprived of step-wise progress feedback. Without r_{bonus}, the agent lacks the local attractor near the reference trajectory and fails to refine tracking in the terminal phase. Both terms are therefore not merely cosmetic shaping; they are essential for training-time convergence and for the final tracking accuracy of the policy.

Note on the energy metric K9. K9 appears lower in every ablation variant than in the baseline (5.314). This does not indicate improved energy efficiency; rather, it is an artifact of earlier episode termination when the agent fails (e.g., collision or excessive orientation error), which truncates the integration window. K9 should therefore only be interpreted jointly with K1, K2, and K4.

Summary. The ablation confirms that W_{ori} and W_p are the two dominant cost components of the composite reward, while the remaining five cost weights contribute refinements in motion quality, base velocity damping, and actuation smoothness. Both positive reward terms (r_{prog} and r_{bonus}) are additionally shown to be load-bearing for training-time convergence and final tracking accuracy. Removing any single secondary cost weight still yields a functioning, if degraded, policy, whereas removing W_{ori}, W_p, or either of the positive reward terms leads to a clearly degraded controller. This supports the reward design choices reported in section IV and shows that the benchmark conclusions do not rely on fine balancing of low-weight terms.

C. PIECEWISE-LINEAR TRAJECTORY

While the previous evaluations focused on tracking a smooth circular end-effector (EE) reference, we additionally tested how well the controller generalizes to linear trajectories. This is relevant for practical on-orbit manipulation tasks, where approach and retreat motions are frequently planned as straight-line segments rather than continuous curves.

Trajectory definition: the linear reference is constructed from the same circle parameters (center and radius) used for the circular benchmark, but the EE is commanded to move along two straight segments connecting three points on that circle: $P_0 = [c_x + r,\ c_y,\ c_z]$, $P_1 = [c_x,\ c_y + r,\ c_z]$, and $P_2 = [c_x - r,\ c_y,\ c_z]$. Using a period T and a switching time $t_1 = T/2$, the piecewise-linear reference position $p_{\text{ref}}(t)$ is

$$p_{\text{ref}}(t) = \begin{cases} P_0 + \frac{t}{t_1}\left(P_1 - P_0\right), & 0 \le t \le t_1, \\ P_1 + \frac{t-t_1}{T-t_1}\left(P_2 - P_1\right), & t_1 < t \le T. \end{cases} \tag{5}$$

The reference is sampled at $T_s = 0.01\,\text{s}$, while the PPO policy is updated at $T_{s,\text{agent}} = 0.1\,\text{s}$ (same as in the other experiments).

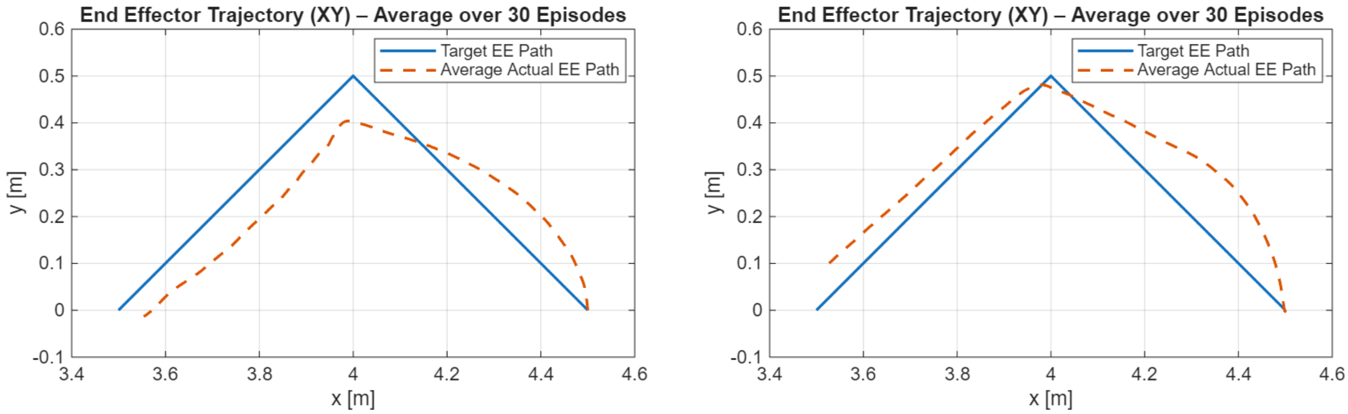
Results: Table 7 reports the KPI averages over $N_{eval} = 30$ evaluation episodes for PPO with the default configuration and the optimized configuration, now tested on the piecewise-linear reference. The optimized PPO improves overall task success (K1 increases from -0.4565 to -0.2977), reduces base attitude disturbance (K4 decreases by $\approx\ 44\%$) (see Figure 15), lowers residual base angular velocity (K8 decreases by $\approx 15\%$), and significantly reduces energy consumption (K9 decreases by $\approx\ 35\%$). The mean and peak end-effector tracking errors (K2 and K3) remain almost unchanged, with a slight improvement in the mean error (see Figure 14). However, the control smoothness metric (K7) becomes worse than the default PPO. This suggests that the hyperparameter optimization (highly beneficial for circular tracking) does not fully transfer to piecewise-linear motion profiles. A straightforward mitigation is to include a mixture of reference types (circular, linear segments, and hybrid trajectories) during training (or during hyperparameter search) to explicitly optimize for trajectory generalization.

Both the circular and piecewise-linear evaluations validate the approach of systematic hyperparameter tuning and network tailoring for safety-critical space robot control, while also revealing that trajectory-specific optimization may be needed for optimal generalization.

In the next section, we will discuss the broader implications of these results, limitations, and future avenues to bring this work closer to deployment on physical testbeds for space robotics.

TABLE 7: Generalization test on a piecewise-linear end-effector trajectory (two-segment ramp). Values are averaged over $N_{eval} = 30$ episodes. Bold indicates the better value according to the KPI definition (K1 higher is better; K2–K9 lower is better).

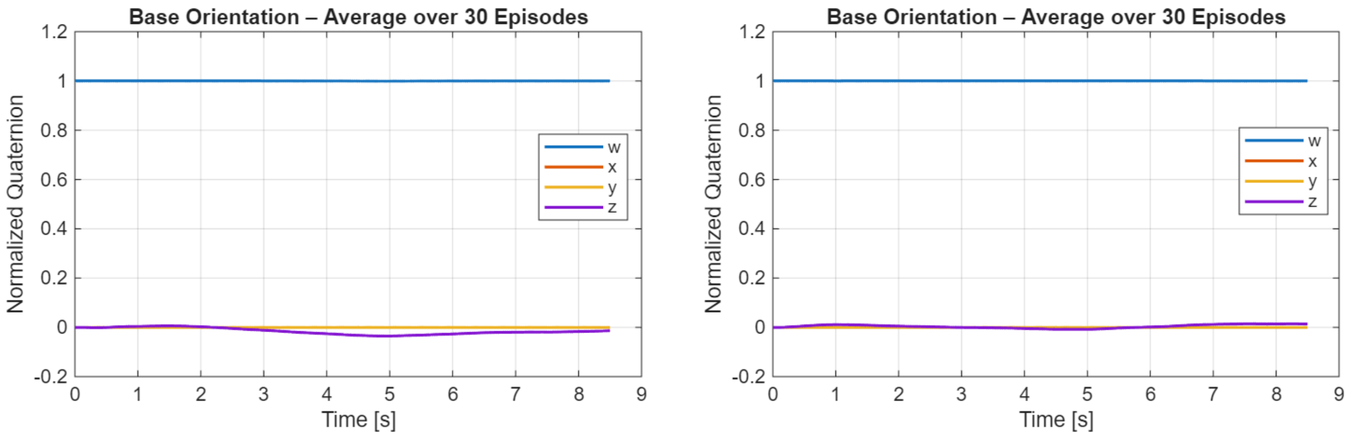
Metric (KPI)	**PPO (Default)**	**PPO (Optimized)**
Mean Episodic Reward (K1)	-0.4565	$\mathbf{-0.2977}$
Mean Squared EE Position Error (K2)	0.0078	**0.0070**
Max EE Position Error (K3)	0.2037	**0.2036**
Mean Base Orientation Error (K4)	0.0412	**0.0231**
Collision Rate (K5)	0	0
Joint Limit Violation Rate (K6)	0	0
Control Smoothness (K7, jerk)	**0.5520**	0.6839
Mean Base Angular Velocity (K8)	0.0399	**0.0339**
Energy Consumption (K9)	7.7144	**5.0341**



(a) PPO (Default)

(b) PPO (Optimized)

FIGURE 14: Target/actual comparison of the ramp end-effector trajectory in the XY plane.



(a) PPO (Default)

(b) PPO (Optimized)

FIGURE 15: Course of base orientation (quaternion components) during a test episode with ramp trajectory.

VIII. DISCUSSION AND CONCLUSION

[R2.14] This work has presented a comprehensive study on applying continuous deep reinforcement learning to the control of a free-floating space robotic arm, with an emphasis on safety and performance. The results demonstrate that an RL-based controller can successfully handle the challenging manipulation task in a space environment. Concretely, the optimized PPO agent achieved a mean squared end-effector tracking error (K2) of 0.0083, a $67.7\,\%$ reduction compared to the default configuration (0.0257), while reducing the mean base orientation error (K4) by a factor of approximately three (from 0.0667 to 0.022) and halving the mean base angular velocity (K8: $0.095 \rightarrow 0.050\,\mathrm{rad/s}$). Crucially, zero safety violations were recorded over all 30 evaluation episodes ($\mathrm{K5} = \mathrm{K6} = 0$), compared to a $5.1\,\%$ joint-limit violation rate for the default agent. Torque smoothness (K7) improved by $48.5\,\%$ and energy consumption (K9) decreased by $16\,\%$. These improvements were achieved simultaneously, indicating a fundamentally more efficient control strategy rather than a trade-off between objectives [1], [22].

[R2.14] A further evaluation was conducted on a piecewise-linear end-effector reference to probe trajectory generalization. The optimized PPO agent improved over the default on most metrics (reward K1: $-0.4565 \rightarrow -0.2977$; base orientation error K4: $-44\,\%$; energy K9: $-35\,\%$), confirming partial transfer of the learned policy. However, the control smoothness metric K7 degraded relative to the default configuration, indicating that the hyperparameter configuration optimized for circular tracking does not fully transfer to piecewise-linear motion profiles. This highlights the need for trajectory-diverse training or trajectory-specific tuning to achieve robust generalization [44].

Practical Implications: The success of PPO (and TRPO) in this domain suggests that policy gradient methods are well-suited for complex, continuous control tasks with coupled dynamics and safety constraints. In particular, PPO's ability to handle continuous action spaces and its stable update rule (clipped surrogate objective) made it robust in the face of the highly nonlinear dynamics of the free-floating robot. The integrated safety components (collision avoidance, etc.) effectively acted as a form of "shielding" during training, they prevented the agent from exploring disastrously unsafe actions by cutting those episodes short [20], [24], [25]. This, combined with the penalty-driven learning, means the final policy inherently respects those constraints (since it learned quickly that violating them yields no reward). For real missions, this is very promising: it implies we can embed safety checks in the training simulator and reasonably expect the trained policy to abide by them when deployed (assuming the real system is within the simulator's fidelity) [24], [31].

[R3.9] **Limitations and Future Work:** Several limitations should be acknowledged. First, a sim-to-real gap remains. Beyond the idealizing assumptions stated in subsection III-C (no sensor noise, no latency, rigid body model, no external disturbances), real deployment introduces additional challenges: (a) domain shift due to inaccuracies in the identified mass and inertia parameters, joint friction, and link flexibility not captured in the URDF model; (b) onboard compute constraints, as the learned neural network policy must execute in real time on embedded space-qualified hardware (typically ARM-based processors with limited floating-point throughput), whereas training was performed on a workstation-class CPU; and (c) sensor-actuator latency, where non-negligible delays between state measurement and torque application can destabilize a policy trained under the zero-latency assumption. Mitigating these effects requires domain randomization over simulation parameters, latency injection during training, and ultimately hardware-in-the-loop (HIL) validation [24], [44]. Second, the RL agent exhibits hyperparameter sensitivity—the tuned PPO configuration is task-specific and may require re-tuning for different trajectories or mission scenarios. Third, the state representation was manually designed; learned representations or recurrent architectures (LSTM) might improve performance under partial observability. Fourth, the safety monitoring covers targeted critical properties but does not constitute exhaustive formal verification of the full mission [23], [24]. Finally, no hardware testing was performed; real-time computation limits, communication delays, and physical interactions remain to be validated on physical testbeds.

Future work should address these gaps through: (i) domain randomization over simulation parameters and latency injection to improve sim-to-real robustness [44]; (ii) extension to more complex tasks such as dual-arm coordination, on-orbit assembly, or active debris removal; (iii) advanced RL architectures (recurrent policies, attention mechanisms) and safe RL algorithms with explicit constraint handling via Lagrange multipliers or barrier functions [24], [25], [31], [32]; and (iv) hardware-in-the-loop validation on our testbed for In-Space Robotized Services (IROS) with virtually simulated microgravity and predicted energy consumption (see fig. 16).

[R2.14] **Conclusion:** In conclusion, this research demonstrates a viable framework for applying deep reinforcement learning to complex space robotic systems. A systematic comparison of six state-of-the-art RL algorithms under identical conditions identified PPO as the top-performing approach for Cartesian end-effector tracking of a free-floating space robot. Subsequent hyperparameter tuning and network tailoring of the PPO agent yielded a policy that simultaneously achieved a 67.7 % reduction in tracking error, a three-fold improvement in base orientation stability, zero safety violations over 30 evaluation episodes, and a 16 % reduction in energy consumption—all relative to the default PPO configuration. The integration of formal safety verification directly into the training loop ensured that these results were obtained under continuously enforced safety constraints, addressing the typical black-box nature of learned controllers. These findings, grounded in quantitative experimental evidence, support the suitability of safety-augmented deep RL for safety-critical space manipulation tasks.

The work also provides insights into the influence of algorithm choice and parameter tuning on learning performance for physical systems. Our findings reinforce the notion that there is no one-size-fits-all in RL, careful selection and tuning of the agent (and network structure) can significantly impact the outcome, even more so than we might expect from generic benchmarks. For practitioners, this means that investing time in benchmarking algorithms (as we did) can pay off in identifying the right approach for the problem at hand.

Finally, while challenges remain for real-world deployment, this study lays a foundation. It suggests that future space robots, tasked with servicing satellites, assembling structures, or handling space debris, could employ learning-based controllers that adapt to complex dynamics and uncertainties, all while operating safely within predefined limits. We envision an architecture where an RL policy drives the robot's motions, supervised by a layer of formal logic that monitors and vetoes any unacceptable actions (though, ideally, the policy has learned not to propose such actions in the first place). Such an approach could provide the flexibility and efficiency of learned behaviors with the trust and guarantee of model-based safety, a combination well-suited to the unforgiving environment of space [1], [25].

Ultimately, our work demonstrates a step in that direction, confirming that with the right algorithms and safeguards, "autonomous learning-based control" can be a powerful paradigm for future space robotic missions. The results encourage continued research at the intersection of robotics, reinforcement learning, and formal methods to further unlock the potential of intelligent space systems.

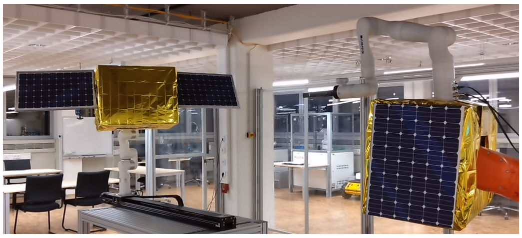


FIGURE 16: Our testbed for In-Space Robotized Services.

ACKNOWLEDGMENT

The authors used DeepL Translator/Write [45], [46] and OpenAI ChatGPT [47] solely for language editing and grammar enhancement. All technical content and conclusions were created and verified by the authors.

REFERENCES

[1] A. Flores-Abad, O. Ma, K. Pham, and S. Ulrich, "A review of space robotics technologies for on-orbit servicing," Prog. Aerosp. Sci., vol. 68, pp. 1–26, 2014, doi: 10.1016/j.paerosci.2014.03.002.

[2] K. Nanos and E. G. Papadopoulos, "On the dynamics and control of free-floating space manipulator systems in the presence of angular momentum," Front. Robot. AI, vol. 4, p. 26, 2017, doi: 10.3389/frobt.2017.00026.

[3] E. Papadopoulos, I. Kaliakatsos, and D. Psarros, "Minimum fuel techniques for a space robot simulator with a reaction wheel and PWM thrusters," in Proc. Eur. Control Conf. (ECC), 2007, pp. 3391–3398.

[4] E. Papadopoulos and A. Abu-Abed, "Design and motion planning for a zero-reaction manipulator," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 1994, pp. 1554–1559, doi: 10.1109/ROBOT.1994.351367.

[5] M. Mühlbauer, M. Chalon, M. Ulmer, and A. Albu-Schäffer, "Software for the SpaceDREAM robotic arm," in Proc. IEEE Aerosp. Conf., 2025.

[6] T. Rybus, "Robotic manipulators for in-orbit servicing and active debris removal: Review and comparison," Prog. Aerosp. Sci., vol. 151, p. 104550, 2024.

[7] E. Guiffo Kaigom, T. Jung, and J. Roßmann, "Optimal motion planning of a space robot with base disturbance minimization," in Proc. 11th Symp. Adv. Space Technol. Robot. Autom., 2011.

[8] E. Guiffo Kaigom, "Deep generative and discriminative digital twin endowed with variational autoencoder for unsupervised predictive thermal condition monitoring of physical robots in Industry 6.0 and Society 6.0," IFAC-PapersOnLine, vol. 59, pp. 7–12, 2025.

[9] Z. Huang, G. Chen, Y. Shen, R. Wang, C. Liu, and L. Zhang, "An obstacle-avoidance motion planning method for redundant space robot via reinforcement learning," Actuators, vol. 12, no. 2, p. 69, 2023.

[10] A. Al Ali and Z. Zhu, "Reinforcement learning for path planning of free-floating space robotic manipulator with collision avoidance and observation noise," Front. Control Eng., vol. 5, p. 1394668, 2024.

[11] M. D'Ambrosio, L. Capra, A. Brandonisio, and M. Lavagna, "Failure management via deep reinforcement learning for robotic in-orbit servicing," in Proc. SPAICE—1st Joint ESA/IAA Conf. AI Space, 2024, pp. 335–339.

[12] A. Al Ali, J. Shi, and Z. Zhu, "Path planning of 6-DOF free-floating space robotic manipulators using reinforcement learning," Acta Astronaut., vol. 224, pp. 367–378, 2024.

[13] Y. Hu, D. Zhou, W. Yao, X. Shao, and G. Sun, "Deep reinforcement learning-based trajectory planning with continuous pose representation for 6-DoF free-floating space robot," Aerosp. Sci. Technol., p. 110540, 2025.

[14] O. Zhang, Z. Liu, X. Shao, W. Yao, L. Wu, and J. Liu, "Learning-based task space trajectory planning framework with pre-planning and post-processing for uncertain free-floating space robots," IEEE Trans. Aerosp. Electron. Syst., early access, 2025.

[15] Y. Wei, X. Bai, and H. Lu, "Trajectory planning of free-floating space robot for non-cooperative tumbling target capture based on deep reinforcement learning," Robotica, vol. 43, pp. 2674–2692, 2025.

[16] Y. Wu, Z. Yu, C. Li, M. He, B. Hua, and Z. Chen, "Reinforcement learning in dual-arm trajectory planning for a free-floating space robot," Aerosp. Sci. Technol., vol. 98, p. 105657, 2020.

[17] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in Proc. Int. Conf. Mach. Learn. (ICML), 2015, pp. 1889–1897.

[18] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in Proc. Int. Conf. Mach. Learn. (ICML), 2018, pp. 1587–1596.

[19] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in Proc. Int. Conf. Mach. Learn. (ICML), 2018, pp. 1861–1870.

[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[21] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Mach. Learn., vol. 8, pp. 229–256, 1992.

[22] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[23] MathWorks, "Simulink Design Verifier," The MathWorks, Inc., Natick, MA, USA, 2024. [Online]. Available: https://www.mathworks.com/help/sldv/

[24] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," J. Mach. Learn. Res., vol. 16, pp. 1437–1480, 2015.

[25] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in Proc. AAAI Conf. Artif. Intell., 2018, pp. 2669–2678.

[26] Open Robotics, "URDF (Unified Robot Description Format)," ROS 2 Documentation, 2024. [Online]. Available: https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/URDF-Main.html

[27] MathWorks, "Import URDF models," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ug/urdf-import.html

[28] MathWorks, "smimport—Import a CAD, URDF, or Robotics System Toolbox model," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ref/smimport.html

[29] MathWorks, "Mechanism Configuration," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ref/mechanismconfiguration.html

[30] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," Int. J. Robot. Res., vol. 32, no. 11, pp. 1238–1274, 2013.

[31] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 22–31.

[32] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in Proc. Eur. Control Conf. (ECC), 2019, pp. 3420–3431.

[33] F. Leutert et al., "AI-enabled cyber–physical in-orbit factory—AI approaches based on digital twin technology for robotic small satellite production," Acta Astronaut., vol. 217, pp. 1–17, 2024.

[34] M. Mühlbauer et al., "Towards an in-orbit cubesat factory," in Proc. 76th Int. Astronaut. Congr. (IAC), 2025.

[35] O. Maqbool and J. Roßmann, "Systems aware scenario engineering for life cycle-spanning verification and validation of complex CPS," Authorea Preprints, 2025. [Online]. Available: https://www.authorea.com

[36] J. Virgili-Llop, "SPART: Spacecraft Robotics Toolkit," 2017. [Online]. Available: https://github.com/NPS-SRL/SPART

[37] E. Ribeiro, R. Mendes, M. Terra, and V. Grassi, "Dynamic parameter identification of a 7-DoF lightweight robot manipulator using probabilistic differential optimization," in Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC), 2022, pp. 1917–1923.

[38] MathWorks, "6-DOF Joint (Simscape Multibody)," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ref/6dofjoint.html

[39] MathWorks, "Spatial Contact Force," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ref/spatialcontactforce.html

[40] MathWorks, "Solver Configuration," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/simscape/ref/solverconfiguration.html

[41] MathWorks, "Fixed-step size (fundamental sample time)," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/simulink/gui/fixedstepsizefundamentalsampletime.html

[42] MathWorks, "Reinforcement Learning Toolbox—rlPPOAgent," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/reinforcement-learning/ref/rl.agent.rlppoagent.html

[43] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in Proc. Int. Conf. Learn. Represent. (ICLR), 2016.

[44] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2017, pp. 23–30.
[45] DeepL SE, "DeepL Translator," [Online]. Available: https://www.deepl.com/de/translator
[46] DeepL SE, "DeepL Write," [Online]. Available: https://www.deepl.com/de/write
[47] OpenAI, "ChatGPT," [Online]. Available: https://chat.openai.com

PATRICK S. ERMISCH Patrick S. Ermisch was born in Frankfurt am Main, Germany, on December 18, 2002. He received the Abitur degree and is currently pursuing the B.Eng. degree in mechatronics at Frankfurt University of Applied Sciences, Frankfurt, Germany, with an expected graduation in 2026. His major field of study is mechatronics.

He has gained practical experience through an internship at SEW-EURODRIVE, where he contributed to the further development of a mobile application for the manual control of logistics robots. In addition, he worked as a tutor for Physics I and Physics II and as a student research assistant, supporting research on particle simulation and the development of an application for visualization and simulation purposes. His research interests include reinforcement learning for robotics, space robotic systems, and simulation-based control methods.

Mr. Ermisch has been listed on the Dean's List of Frankfurt University of Applied Sciences multiple times in recognition of his academic performance.

ERIC GUIFFO KAIGOM was born in Douala, Cameroon. He received the Dipl.-Ing. and Dr.-Ing. (summa cum laude) degrees in electrical engineering from RWTH Aachen University, Aachen, Germany, in 2010 and 2016, respectively. He was a Research Associate with the Institute for Man-Machine Interaction at RWTH Aachen University, where he led the "Intelligent Robots" team and coordinated technology development and transfer activities with partners in robotized manufacturing and on-orbit servicing, including projects within Horizon 2020 EU and In-Space Automation programs. He then worked in the industry as a Head of Emerging Technologies, serving e.g. as Co-PI for the design and acquisition of a data- and AI-driven flagship project. He is currently a Full Professor with the Frankfurt University of Applied Sciences, Frankfurt, Germany. His research interests include Robotics, Digital Twin, and AI/ML. Dr. Kaigom serves the academic community, currently e.g. as an Associate Editor for IEEE Transactions on Technology and Society and reviewer for several journals (IEEE Trans. Ind. Inform., IEEE Trans. Cybern., Journal of Manufacturing Systems, Engineering Applications of Artificial Intelligence, IEEE RA-L, etc.). He is a recipient of best paper and academic awards, including the Borchers' Medal of the RWTH Aachen University.